



Integración

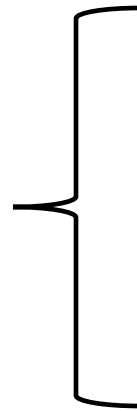
- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Reuso de software

Es útil tener piezas de software que sirven para más de un propósito (teniendo cuidado con su dominio)

Se puede identificar ese proceso:

Abstracción
Selección
Especialización
Integración



Lenguajes de Alto y Muy Alto Nivel
Copy & Paste
Componentes Esquemas de software
Generadores de aplicaciones
Transformaciones
Arquitecturas de Software

Motivación: Heterogeneidad

- Los sistemas de hoy en día **no pueden ser un único bloque para siempre**
- Diferentes formatos, semántica de los datos, lenguajes y plataformas de Software y Hardware
- Muchas Aplicaciones objetivo diferentes:
 - ERP (*Enterprise Resource Planning*)
 - SAP
 - PDM (*Product Data Management*)
 - SCM (*Software Configuration Management*)
 - B2B
 - Etc...



Motivación: AIM

- Aplicaciones de Integración y Middleware
- Los **costos de integrar** han ido creciendo:
 - \$6.4 Billones en el 2005
 - \$11 Billones en el 2007
 - ~28.5 Billones 2017
 - Pérdidas por mala integración (CAD) \$1 Billón (2002)
- Diversos factores que lo han afectado:
 - Diferentes empresas
 - Data operacional
 - Data no estructurada
 - AIM incompatible
- No solo con sistemas externos



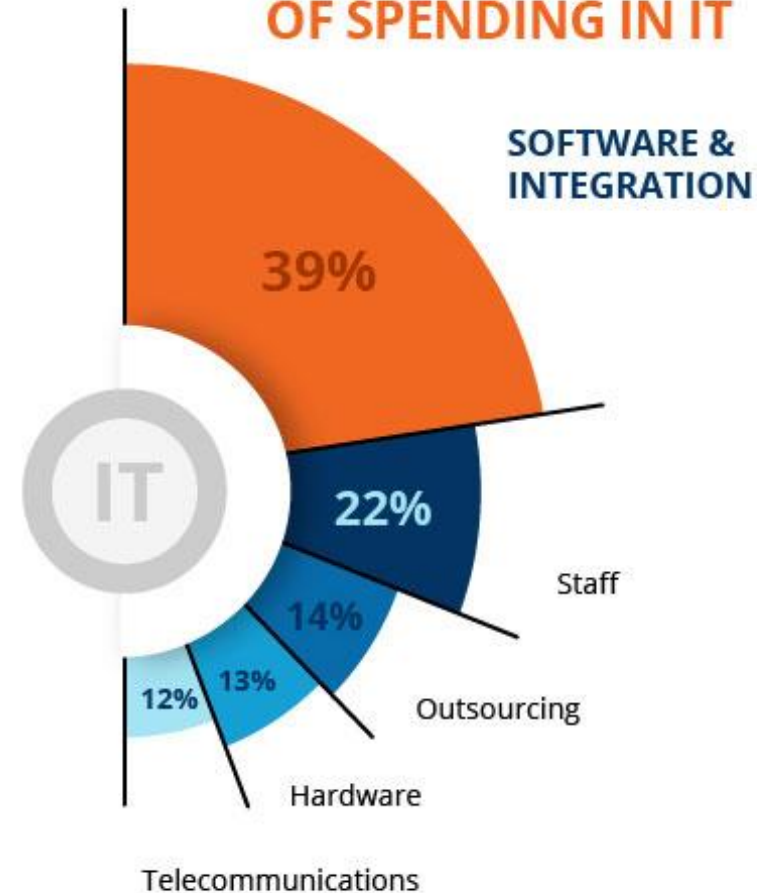
\$3.69 Trillion

Were spent worldwide
on IT in 2020

In 2022 it should pass
the four trillion mark.

Only three countries in
the world have a GDP
above this number, US,
China and Japan.

DISTRIBUTION OF SPENDING IN IT





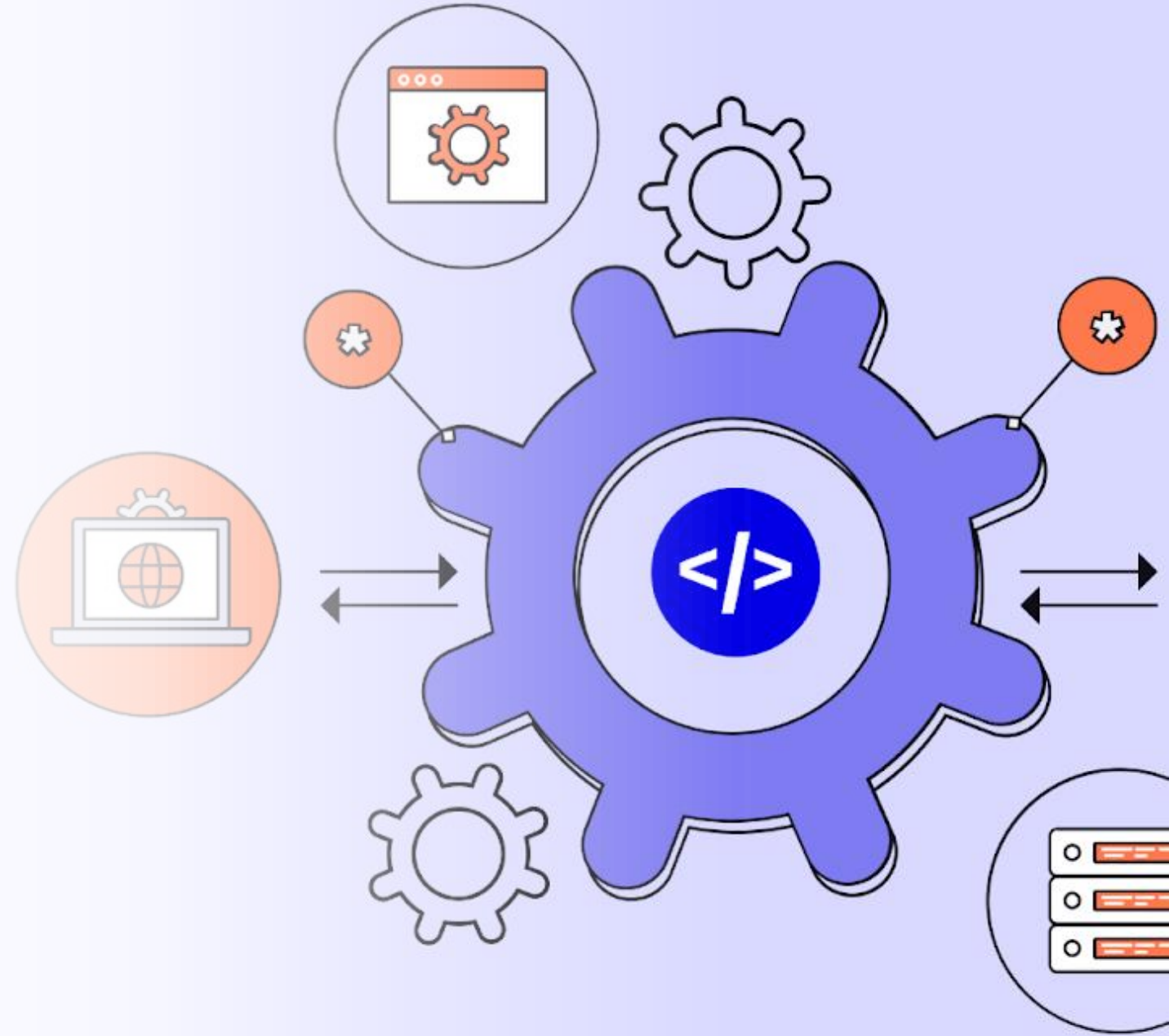
Se puede elevar a ser **más de 788** para los casos de **empresas grandes** con más de 50.000 empleados

Integración

Esfuerzo requerido para integrar una aplicación en un contexto mayor (Bass, Kazman)

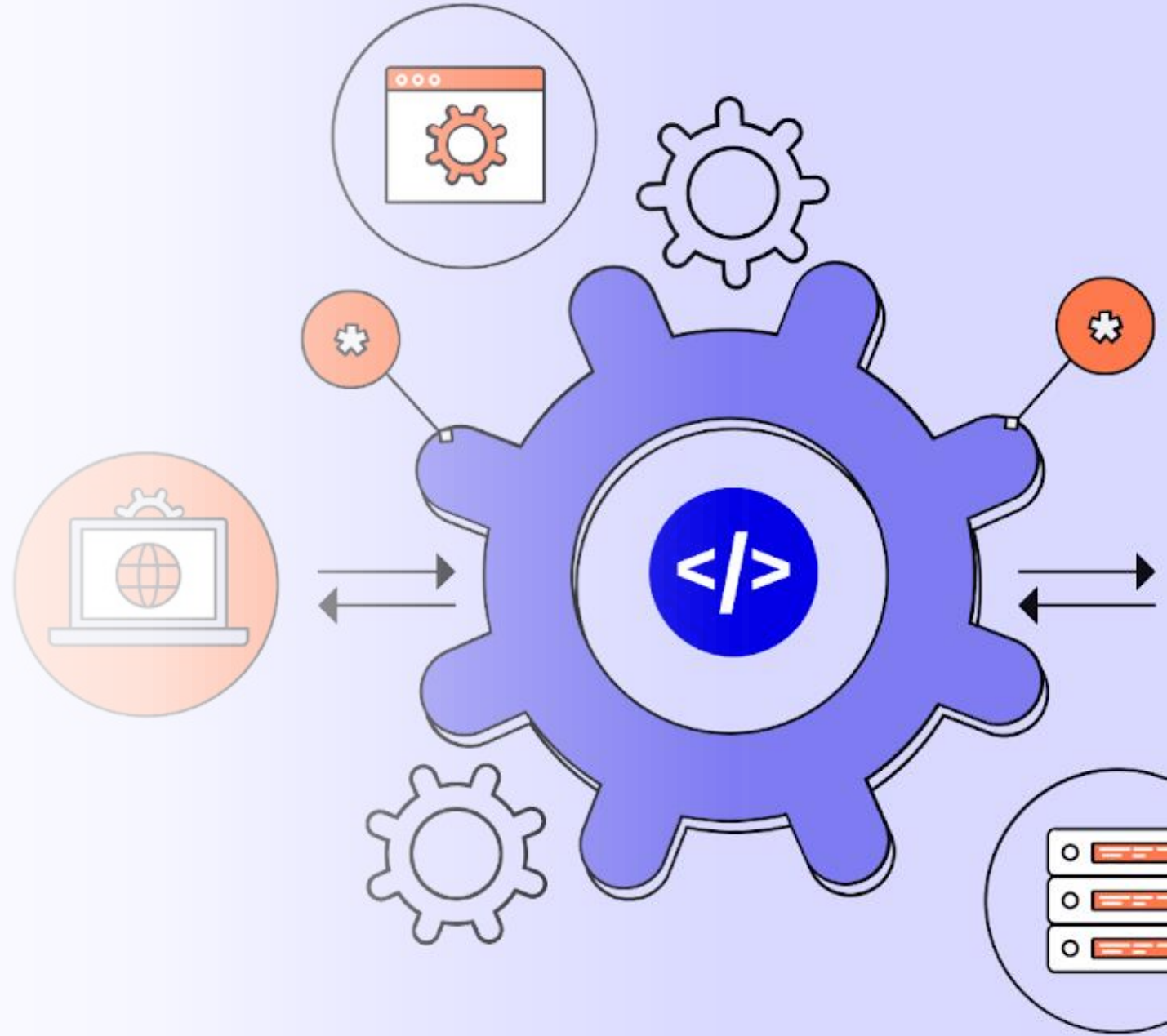
Objetivos:

- Manejo apropiado del acoplamiento en el software
- Potenciar uso/reuso del software
 - Con mucho cuidado
- Potencial evolutivo
- Implementación de estilos más complejos



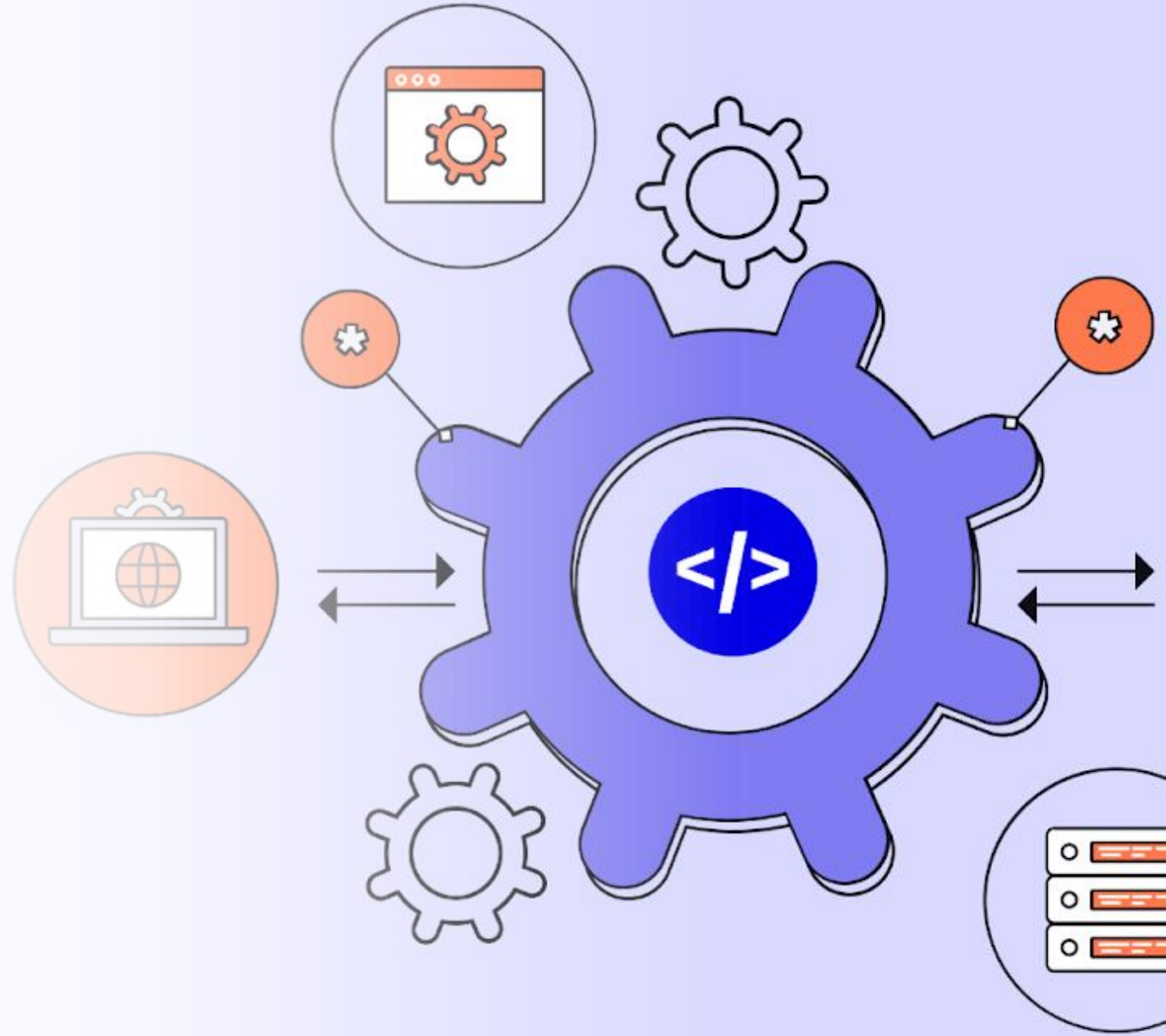
¿Cómo implementar integrabilidad?

- Discernir
 - Distancia entre elementos
 - *Connascence*
 - Capacidad de control
- Actuar
 - Acciones para añadir nuevos componentes
 - Como implementar nuevas versiones
 - Observar/adaptar estándares
- Preferencia en minimizaciones de costos
 - LOC
 - Tiempo
 - Recursos
 - HH/FTE



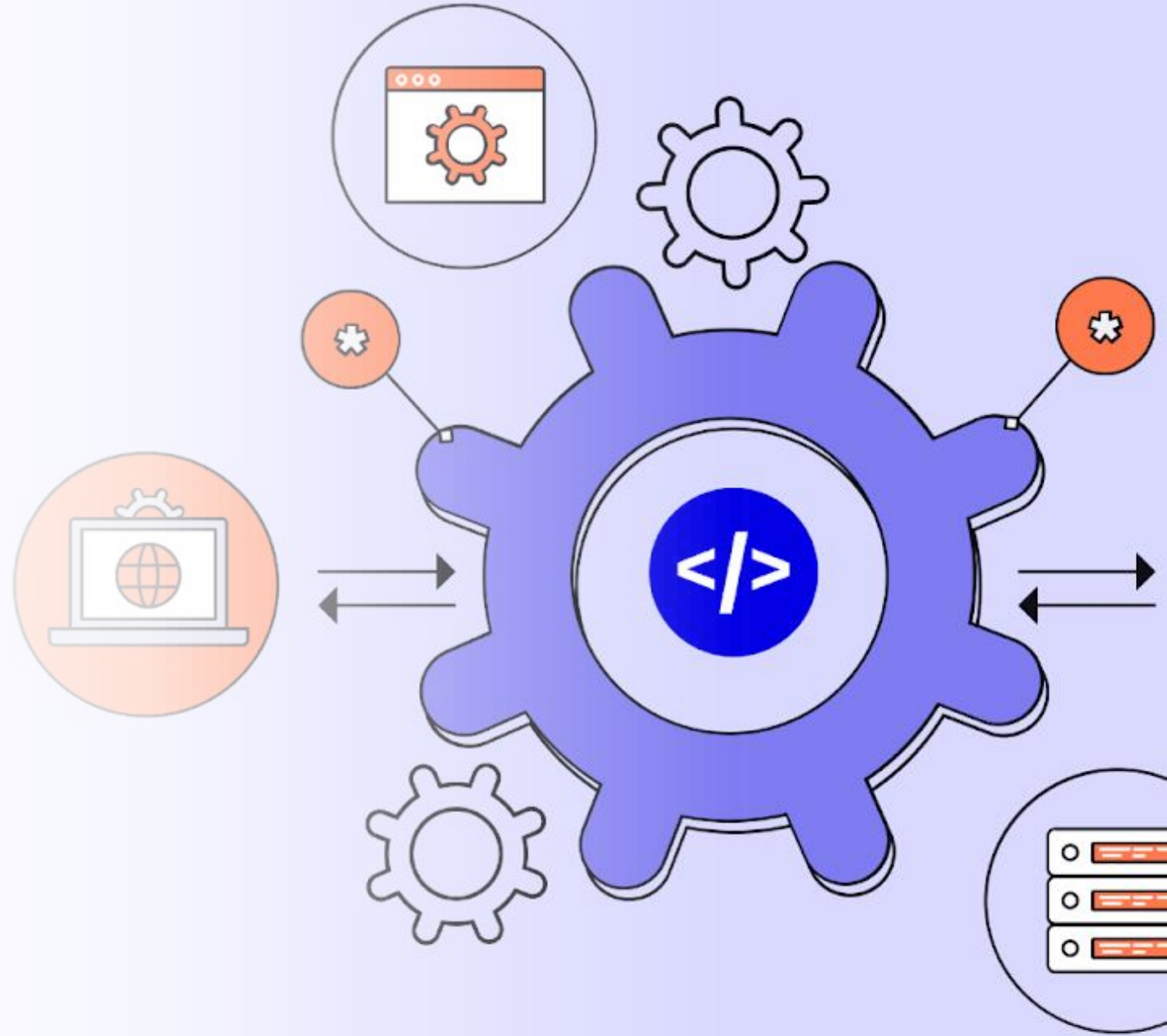
Conceptos básicos de integración

- Distancia modular
- Encapsulamiento
 - Principio de ocultamiento de información
 - Aislación de significado
- DDD (puede estar parcialmente)
 - *Bounded context*
 - *Aggregate*
 - *Ubiquitous language*

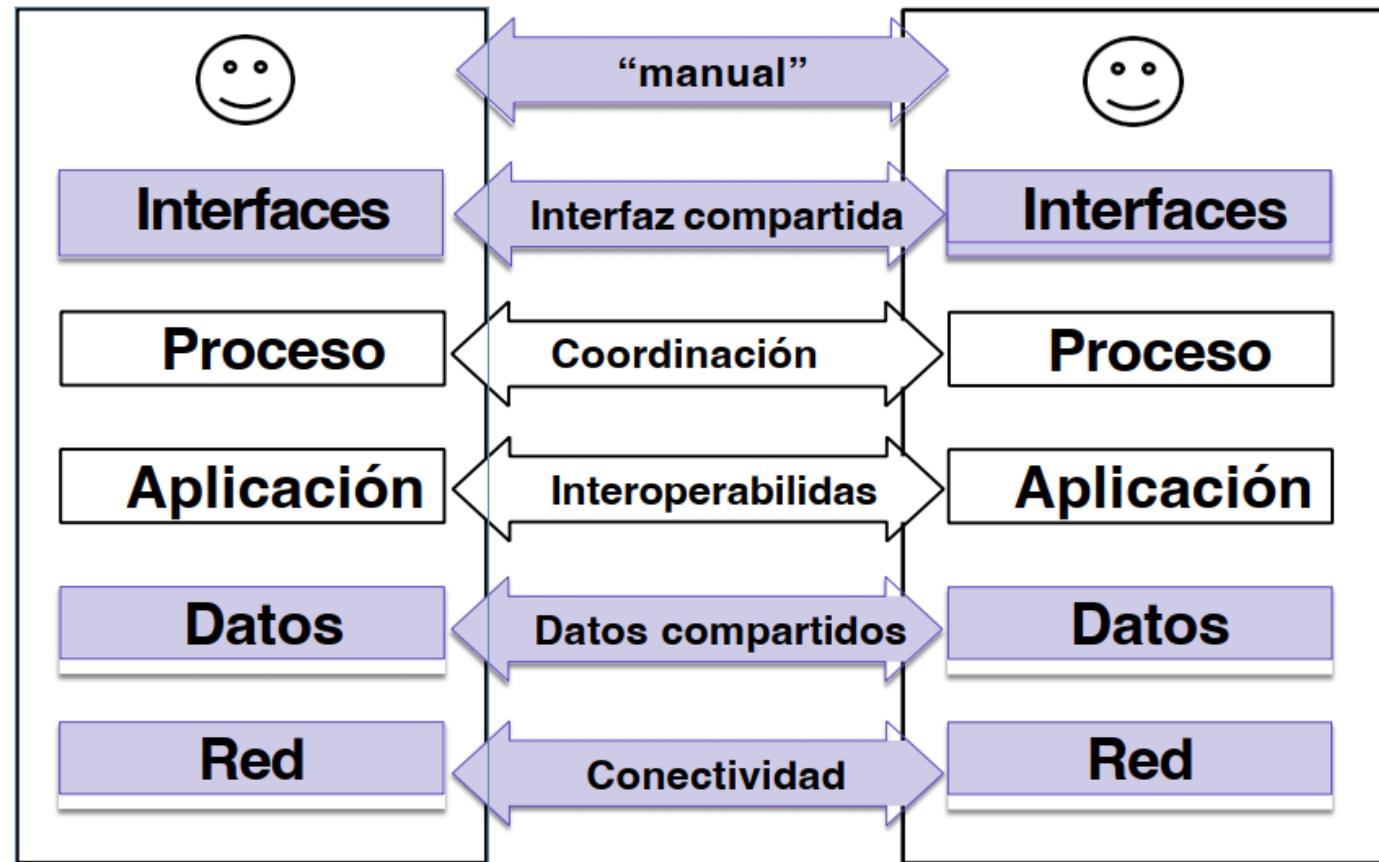


Principales modelos de Integración

- **Common/Content:** A nivel de **datos** (disponibilizar DB a otras aplicaciones):
 - Flexible, simple, pero **peligrosa**
- **Domain/Contract:** **API** (*Application Programming Interface*):
 - Funciones (parametrizadas) que controlan el acceso a los datos. Garantizan integridad de datos y reglas de negocio
 - Compleja, costosa, pero **segura**
- **Middleware-based:**
 - Software especializado en integración

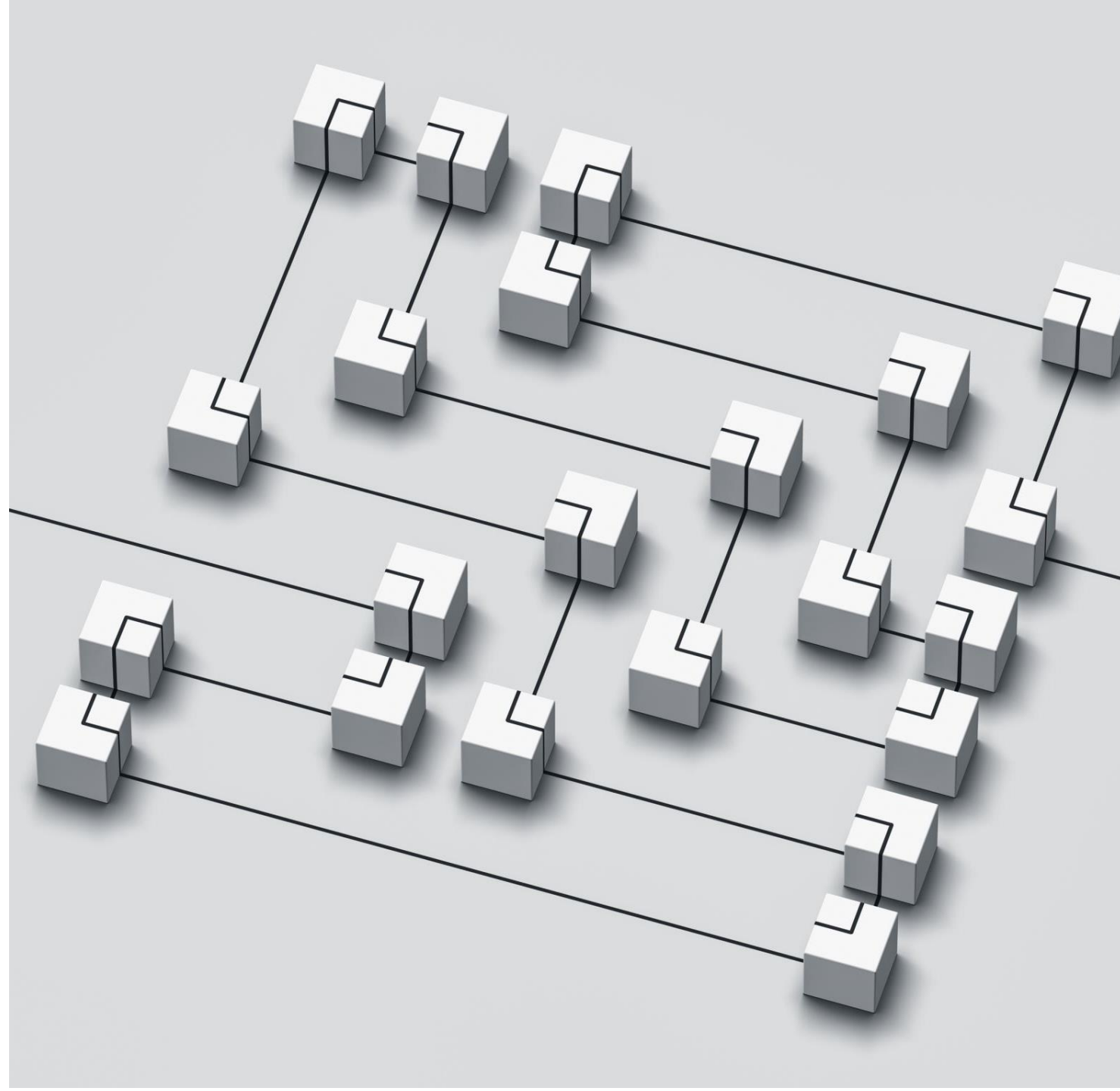


Niveles comunes de integración



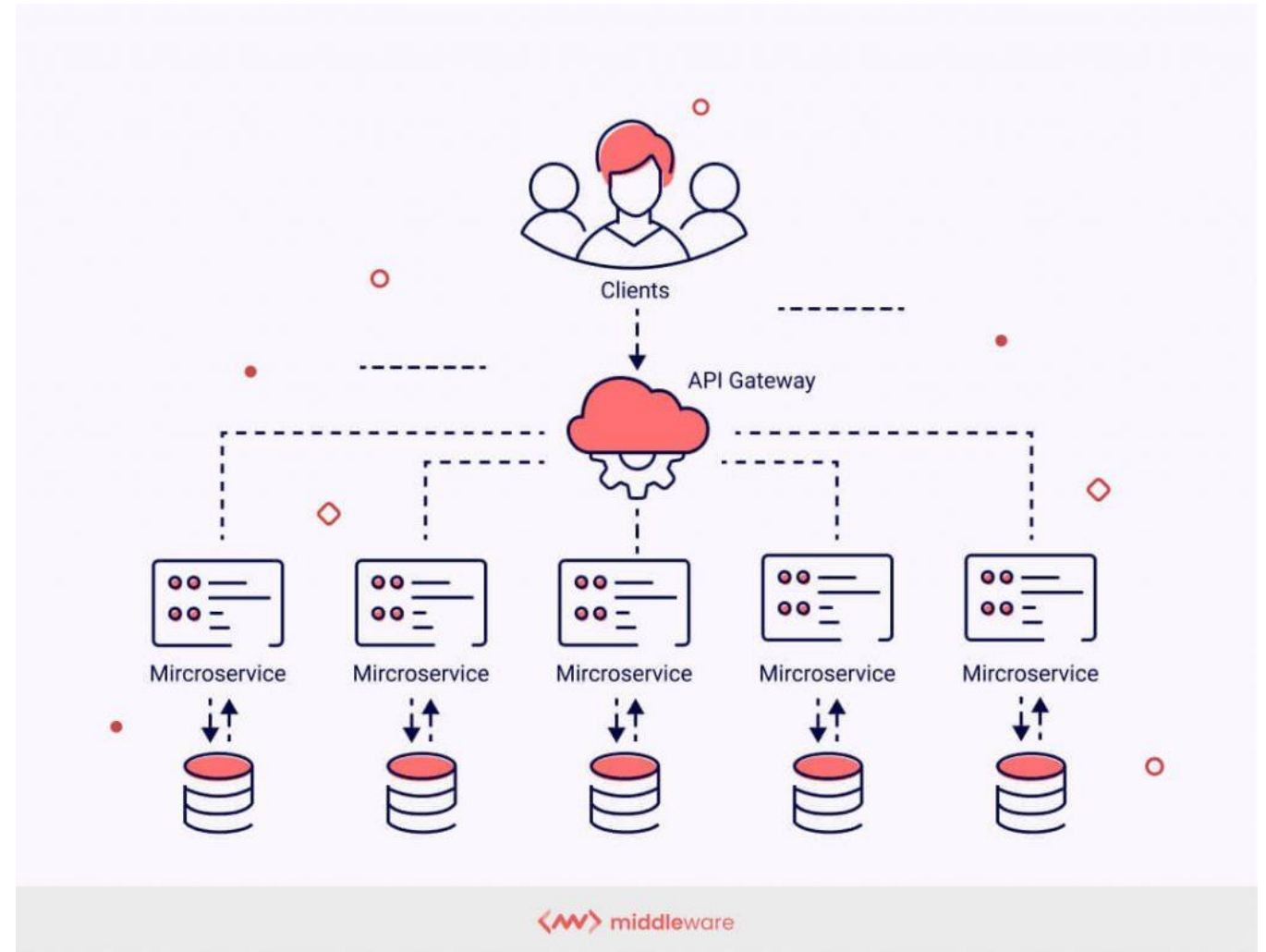
Integrando sistemas: Patrones/Tácticas

- Soluciones reusables capaces de **conectar** dos o más sistemas
- **Lenguaje compartido**
 - Muy similar a los estilos y patrones de diseño
- **Simplifican implementaciones** y resuelven problemas ya comunes



Integrando sistemas: Patrones

- Básicos
 - Encapsular
 - Middleware
 - Estándares
- MOM
 - Brokers
 - Motores de procesos
- SOA
- APIs estructuradas
 - GraphQL
 - REST
- RPC
 - gRPC

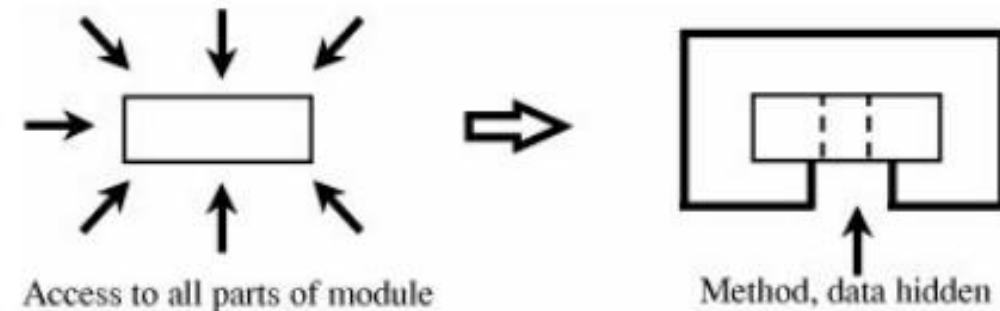


Encapsulamiento

Envolver **un módulo en una interfaz explícita** y regular todos los accesos
(Bass, Kazman)

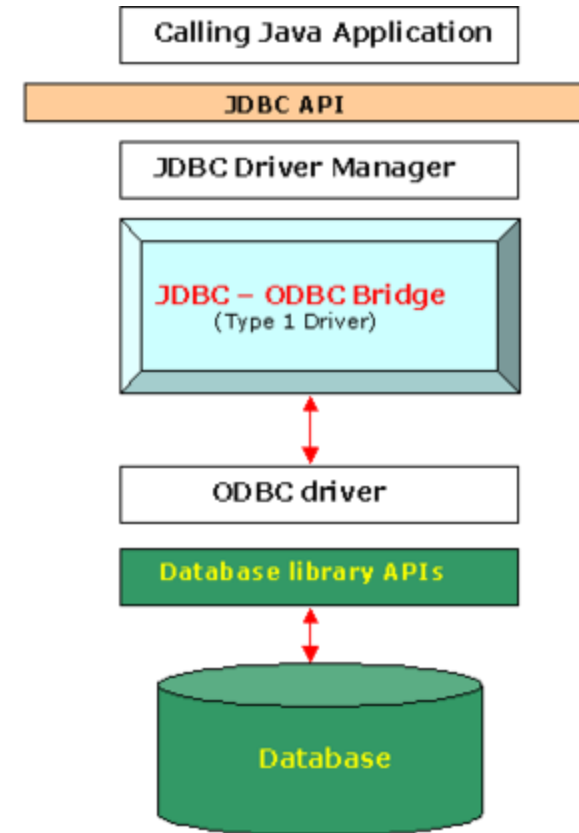
- Permite **reuso cómodo** de funcionalidad
 - Capacidades de Testing y seguridad
- Principio de **ocultamiento de información**
 - Evita acoplamiento indeseado
 - Uso de interfaces (contratos) para comunicaciones
 - Evita propagación indeseada de cambios
- Surgimiento del **concepto de API**
 - *Application Programming Interface*

Figure 3-18. Information Hiding.



Middleware

- **Software intermediario** para resolver dependencias
 - Provee servicios y capacidades comunes a aplicaciones que pueden o no estar bajo nuestro control
- Capa intermedia entre sistemas
 - Ojo que **no fuerza un sistema por capas**
- Facilita **integración**
 - Mejora abstracción e interoperabilidad
- Ejemplos:
 - MOM
 - JDBC
 - Reverse proxy
 - API Gateway



Retomemos el concepto del
monolito



Monolito: una primera arquitectura

- Sistema contenido en **una sola quanta**
- Un bloque de software, **una base de datos**
- Alta cohesión, acoplamiento estático, aunque aprecia modularidad
- Implicancias dependientes del estilo, no es inherentemente malo (debatible)



Monolito: ¿Es la única opción para partir?

- No necesariamente
 - Muy útil para ambientes de desarrollo rápido
 - Trade-off de tener una única quanta
- No hay preocupación por transmisiones de datos
 - ¿Es siempre lo mejor?
- En su proyecto ya están tomando una **arquitectura más distribuida**
 - ¿Cómo coordinamos todo?



Sin monolito: Comunicación “indirecta”

- No podemos asumir comunicación punto a punto cuasi instantáneas
 - No ignorar principios de integrabilidad
- Hay que pensar formas para generar conexiones simples y reusables
- La mejor separación dependerá de nuestro contexto
 - Pero mientras más fácil sea integrar los subsistemas, nos será más fácil tomar esa decisión



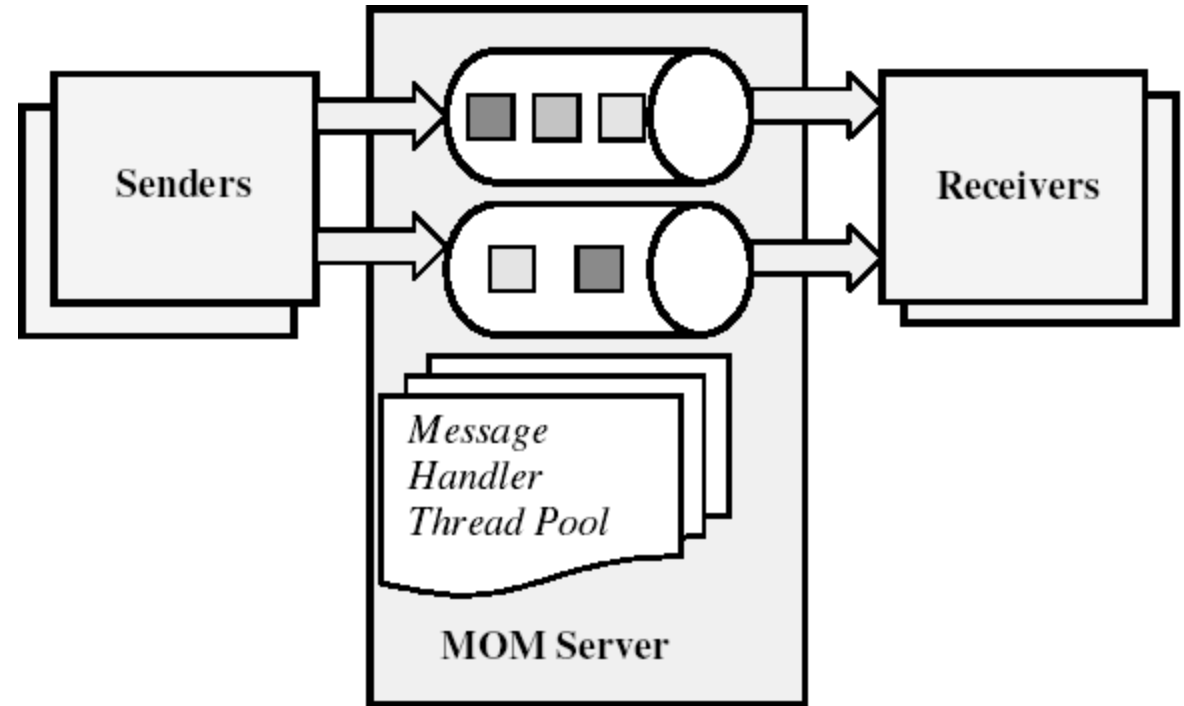


Integración II

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

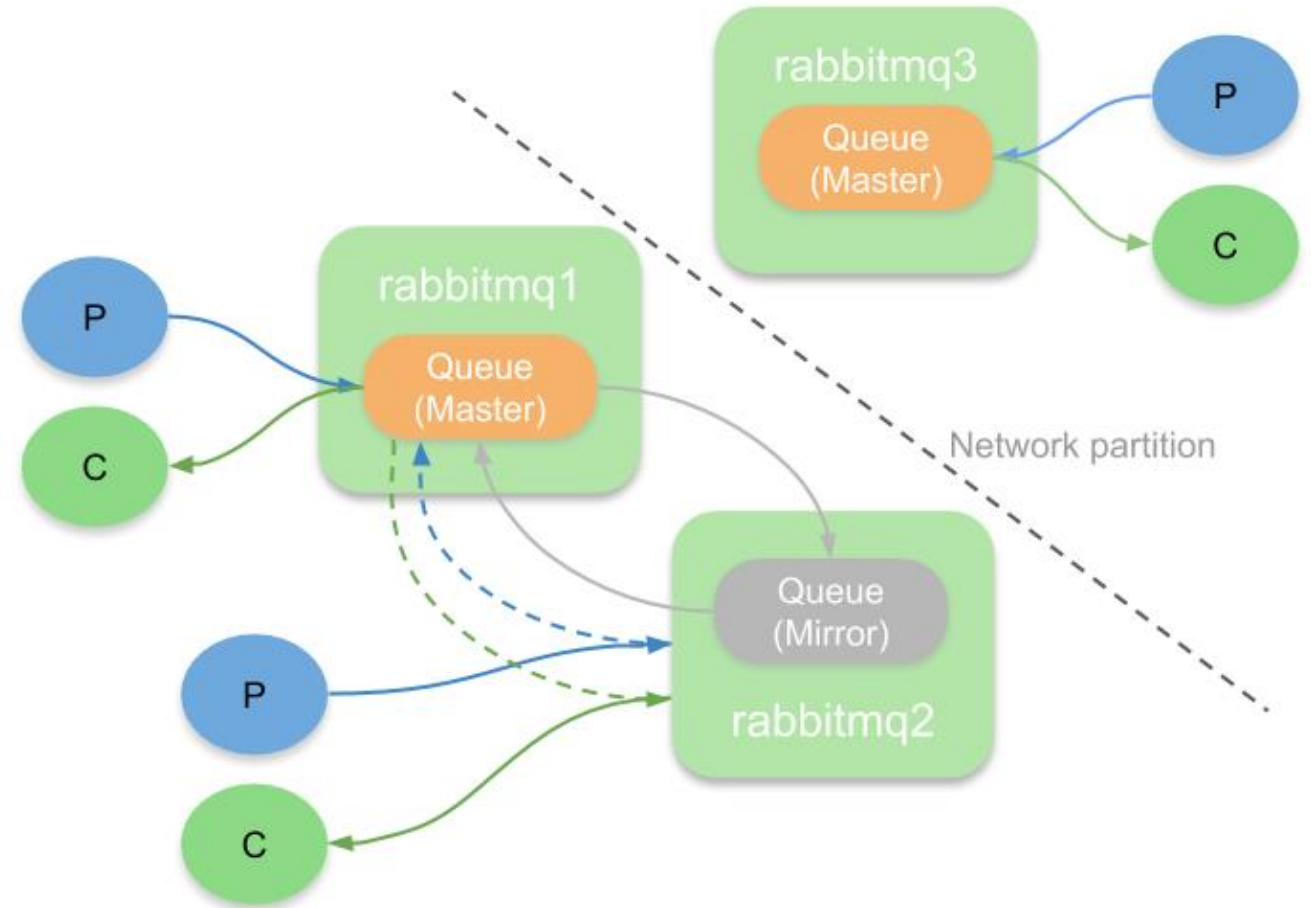
Message Oriented Middleware (MOM)

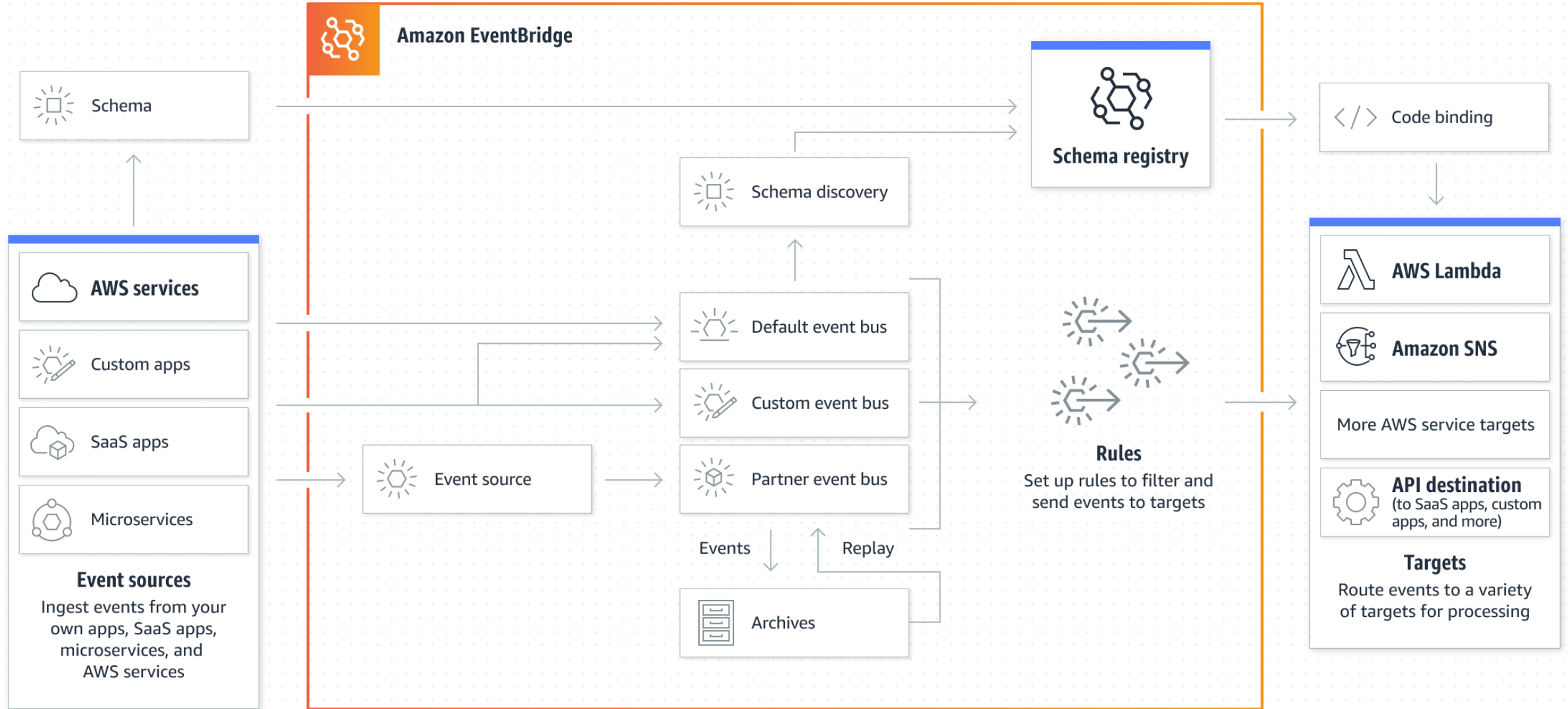
- El middleware es un servidor de colas entre clientes que toman los roles de *sender* y *receiver*
 - *Fire and forget*
- Priorizaciones
 - Escalabilidad
 - Disponibilidad
 - Tolerancia a fallos
- Cons
 - Formato de datos acordado
 - Complejidad en el intercambio de mensajes



Mejorando escalabilidad de MOM: Pub/Sub

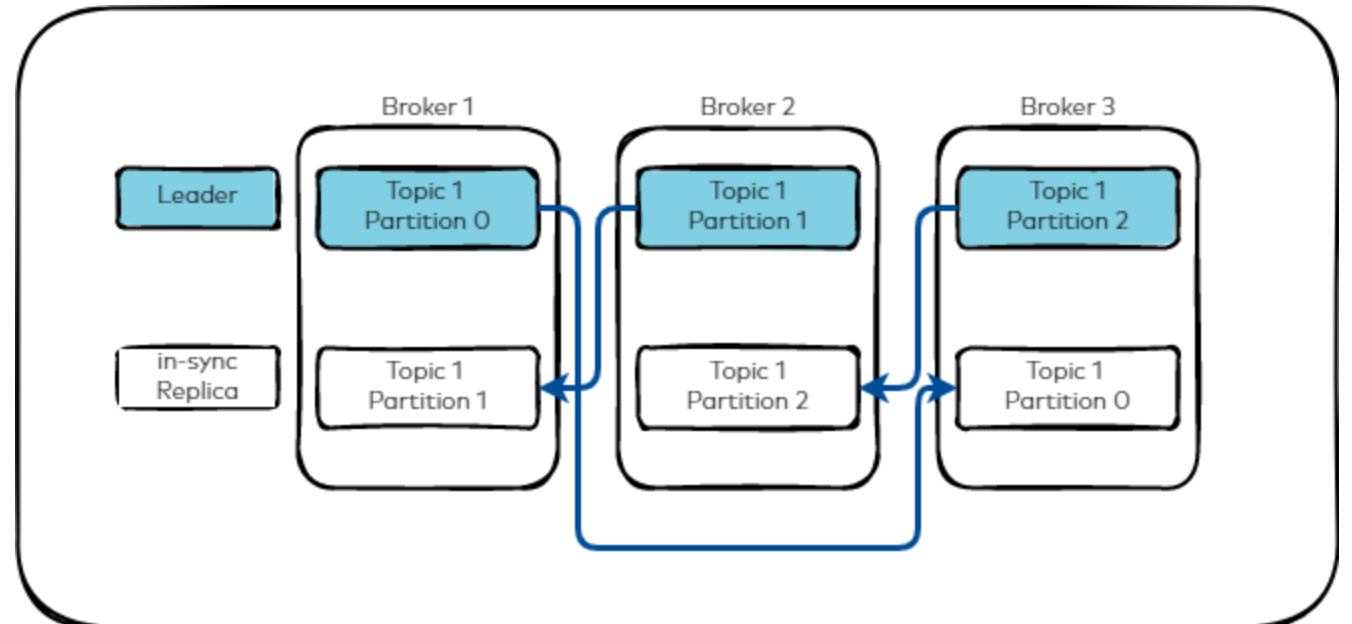
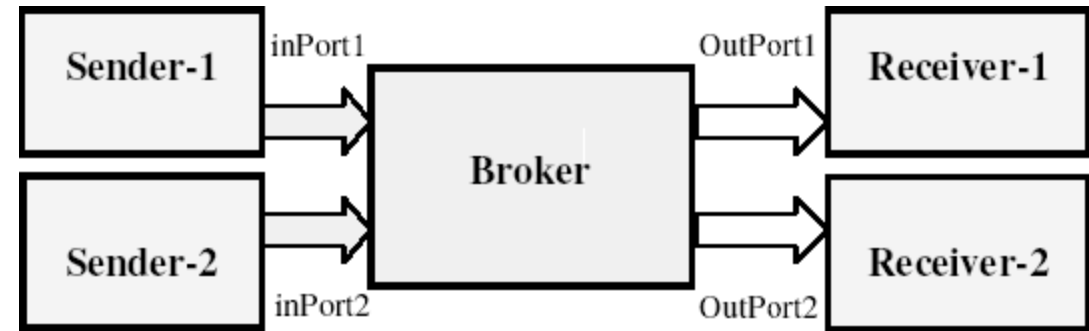
- Implementar patrones de resiliencia
 - *Mirroring*
 - Distribucion
 - *Quorum*
- Mensajería
 - Asincronía
 - Orientación 1-N , N-N





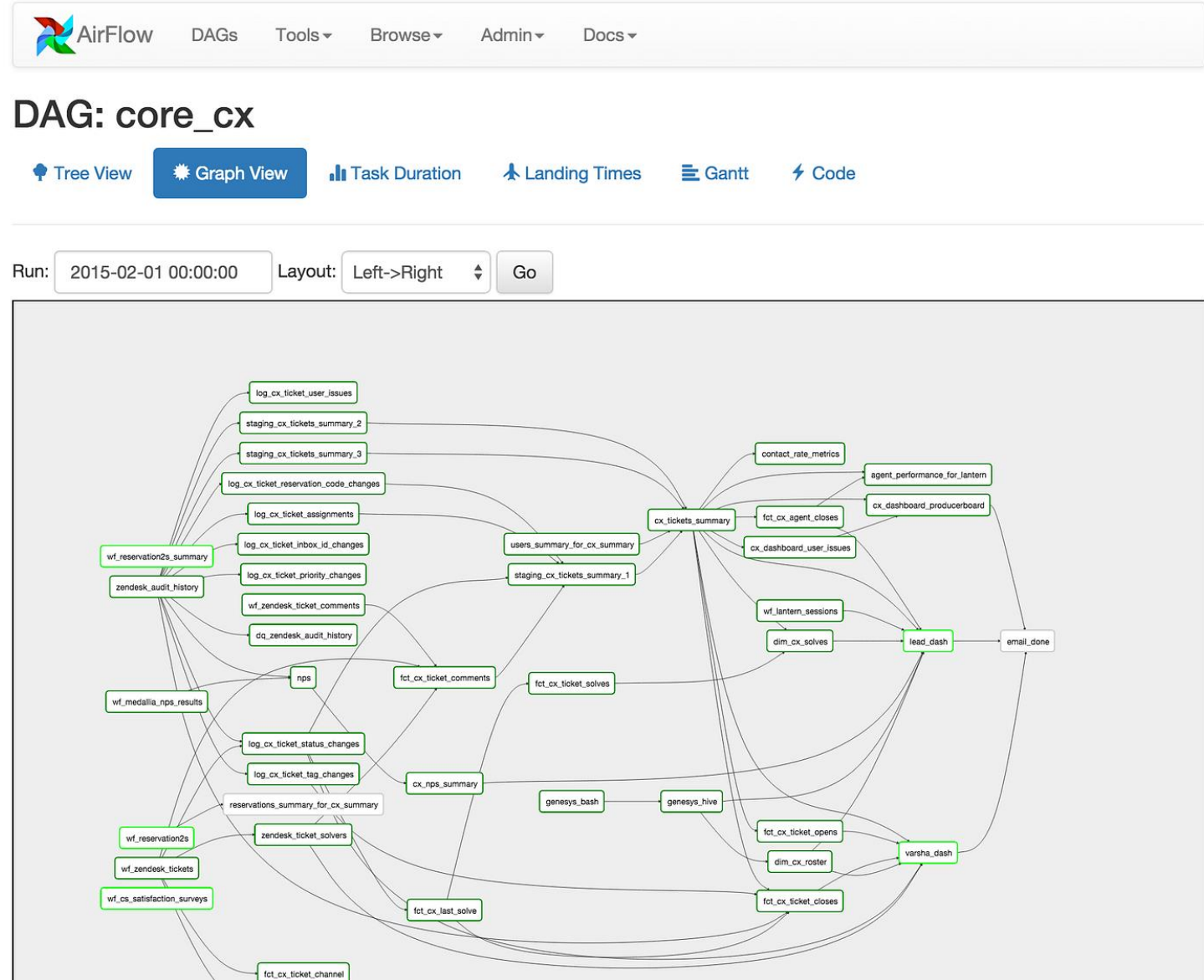
Mejorando escalabilidad de MOM: Pub/Sub

- El Broker es un **intermediario** entre los *senders* y los *receivers*
- 2 Responsabilidades clave
 - Ruteo: El Broker conoce a que receptor debe despachar el mensaje recibido
 - Transformación: El Broker encapsula reglas de transformación de mensajes
- Alta escalabilidad
- ¿Dónde usar?
 - Sistemas de bajo acoplamiento
 - EDA y otros



Motores de procesos

- *Superset* del broker
- Maneja estado de proceso
- Vincula pasos de lógica de negocio con sistemas de software
- Define control de flujo macro
- Pros
 - Encapsulamiento de procesos
- Cons
 - Difícil de escalar (persistencia)
 - SPOF

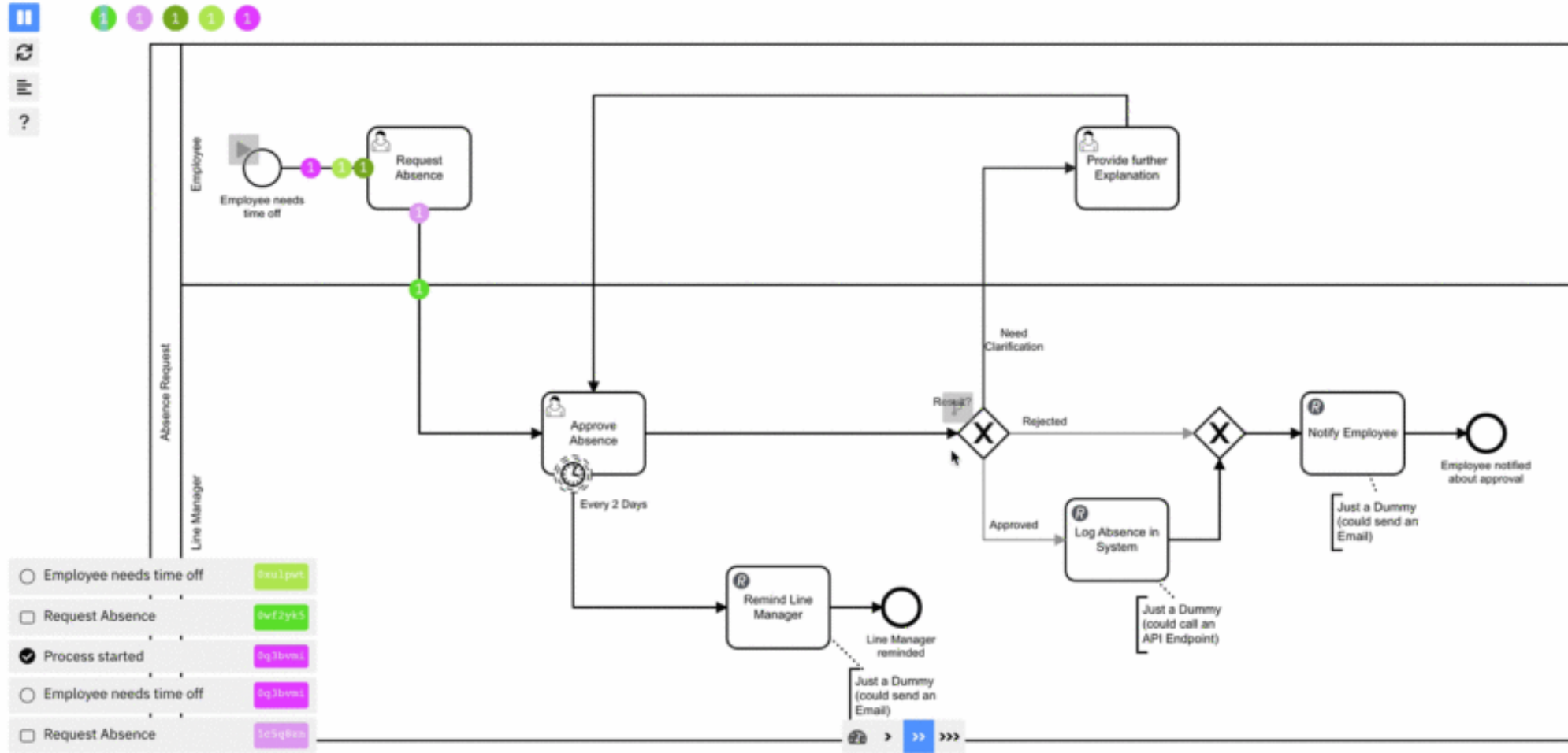


Token Simulation

Test process

Deploy diagram

Start instance



Problems 0

Check problems against: Zeebe 8.0



SOC/SOA

- SOC (*Service Oriented Computing*)
 - Paradigma que usa el servicio como idea básica para permitir el desarrollo de aplicaciones interoperables, masivamente distribuidas, capaces de evolucionar, de manera rápida y a bajo costo
 - Un servicio es un mecanismo que permite acceso a una o más funcionalidades de negocios bien definidas bajo el control de diferentes propietarios
- SOA (*Service Oriented Architecture*)
 - Estilo arquitectónico que sigue la línea de SOC y permite la visibilidad de los servicios y su descripción, la descripción de los efectos del servicio, su ejecución, políticas e interacción

REST

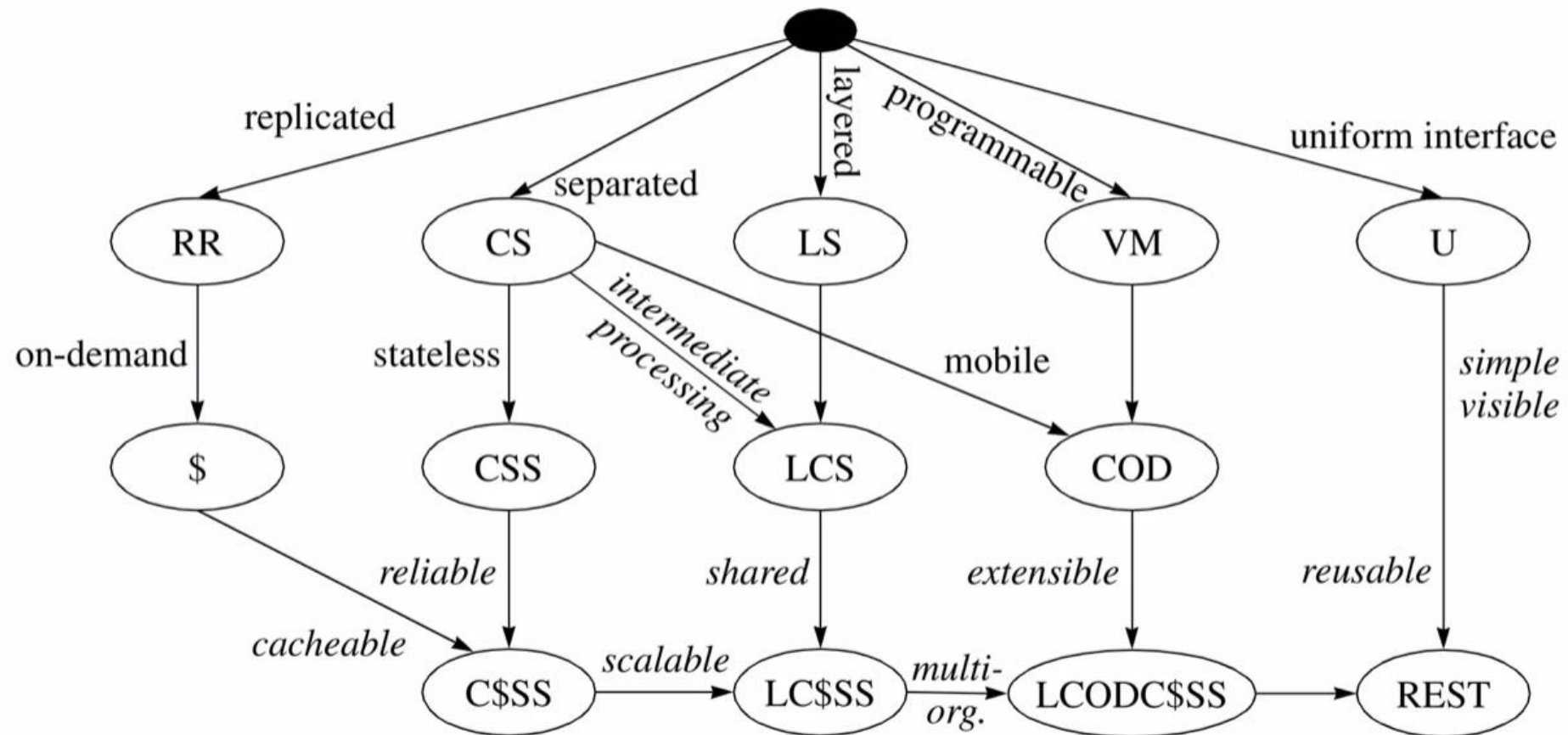
- *Representational State Transfer*
- Roy Fielding, 2000
 - Autor principal de: HTTP/1.1 - 1999
 - Autor secundario: URI Syntax - 1998
- Conjunto de **restricciones arquitecturales** que subyacen a la Web
- Define estilos arquitectónicos y el **Diseño de Arquitecturas basadas en redes**

Diseño de REST

Fielding evaluó **calidad vs. Estilos arquitectónicos** para diseñar HTTP (y la Web)

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
PF			±			+	+	+		+	+			
UPF	PF	-	±			++	+	+		±	++	+		
RR			++		+									+
\$	RR		+	+	+	+								
CS					+	+	+							
LS			-		+		+				+		+	
LCS	CS+LS		-		++	+	++				+		+	
CSS	CS	-			++	+	+					+		+
C\$\$\$	CSS+\$	-	+	+	++	+	+					+		+
LC\$\$\$	LCS+C\$\$\$	-	±	+	+++	++	++				+	+	+	+
RS	CS			+	-	+	+					-		
RDA	CS			+	-	-						+		-
VM						±		+				-	+	
REV	CS+VM			+	-	±		+	+			-	+	-
COD	CS+VM		+	+	+	±		+		+		-		
CODC\$\$\$	LC\$\$\$+COD	-	++	++	+4+	±±	++	+		+	+	±	+	+
MA	REV+COD		+	++		±		++	+	+		-	+	
EBI				+	--	±	+	+		+	+	-		-
C2	EBI+LCS		-	+		+	++	+		+	++	±	+	±
DO	CS+CS	-		+			+	+		+	+	-		-
BDO	DO+LCS	-	-				++	+		+	++	-	+	

“Árbol” de derivación de REST



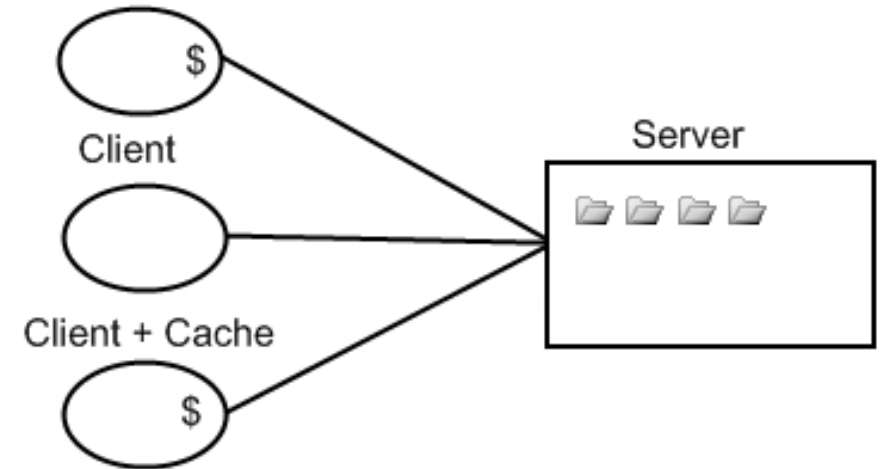
Propiedades deseables de mantener en la Web

- Barrera de entrada baja (autores, editores, lectores)
- Extensibilidad
- Hipermedia distribuida
- Escalabilidad masiva y anárquica
- *Deployment* independiente

“Estilos” arquitectónicos que influyen REST

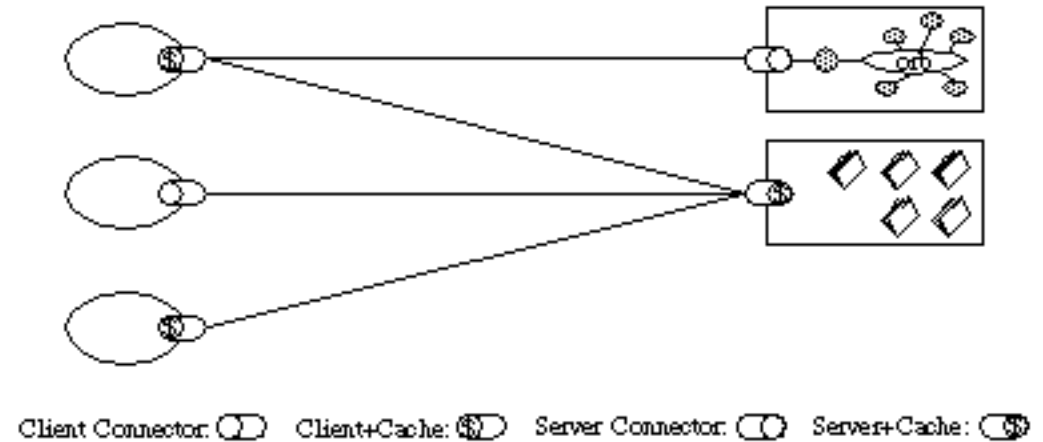
- En Capas

- CS: Client Server
- LCS: Layered Client Server
- CSS: Client Stateless Server
- C\$SS: Client-Cached Stateless Server
- LC\$SS: Layered Client-Cached Stateless Server
 - Añade proxy y gateway (Ej. DNS)



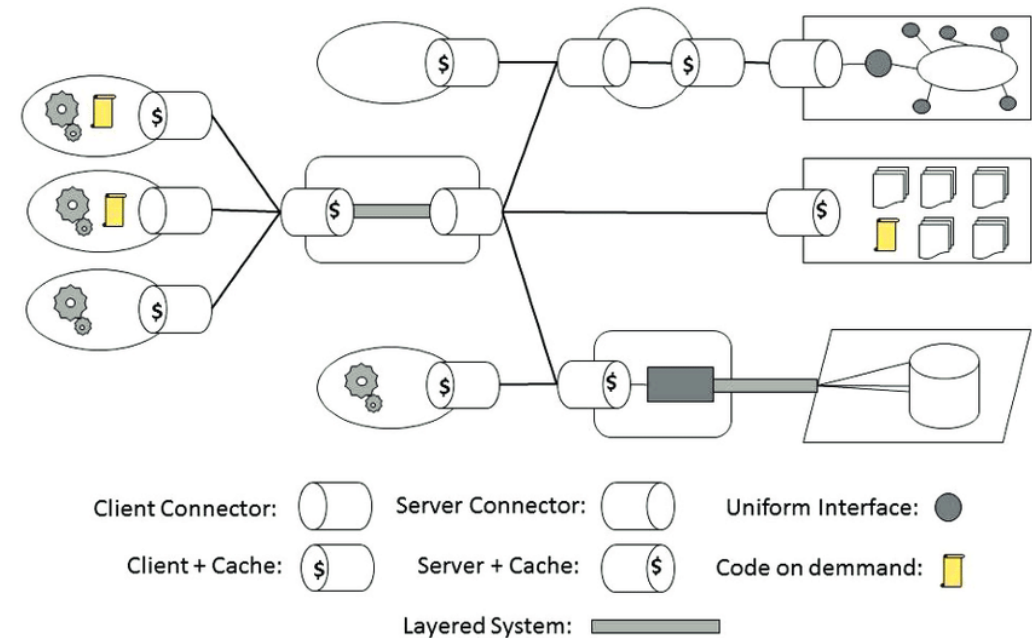
“Estilos” arquitectónicos que influyen REST

- **Interfaz Uniforme** Entre Componentes (Restricciones)
 1. Identificar recursos (URI)
 2. Manipular recursos mediante representaciones (HTML)
 3. Mensajes auto-descriptivos (HTTP)
 4. Hypermedia como Motor de Estado de Aplicaciones (HATEOAS)



“Estilos” arquitectónicos que influyen REST

- **Replicación**
 - RR: Repositorio Replicado
 - \$: Caché
- **Código Móvil**
 - VM: Máquina Virtual
 - COD: Código bajo demanda
 - Un cliente solicita un código que se ejecuta localmente (ej. JavaScript)
 - LCODC\$SS: Layered, Code On Demand, Client-Cached Stateless Server



Propiedades de Calidad que REST considera

- Network **Performance**
 - Throughput
 - Overhead
 - Bandwidth
 - Bandwidth usable
- Network **Efficiency**
 - Minimizar el uso de la red (caché, replicar data, uso offline)
- User **perceived performance**
 - Latencia
- **Confiabilidad**
 - Redundancia, evitar puntos únicos de fallo, monitoreo



Propiedades de Calidad que REST considera

- Escalabilidad
 - Distribuir servicios entre componentes
 - Simplificar componentes
- Simplicidad
 - Separación de responsabilidades
- Visibilidad
 - De las interacciones y mejoras
- Portabilidad
 - Incluye mover código
- “Modificabilidad”



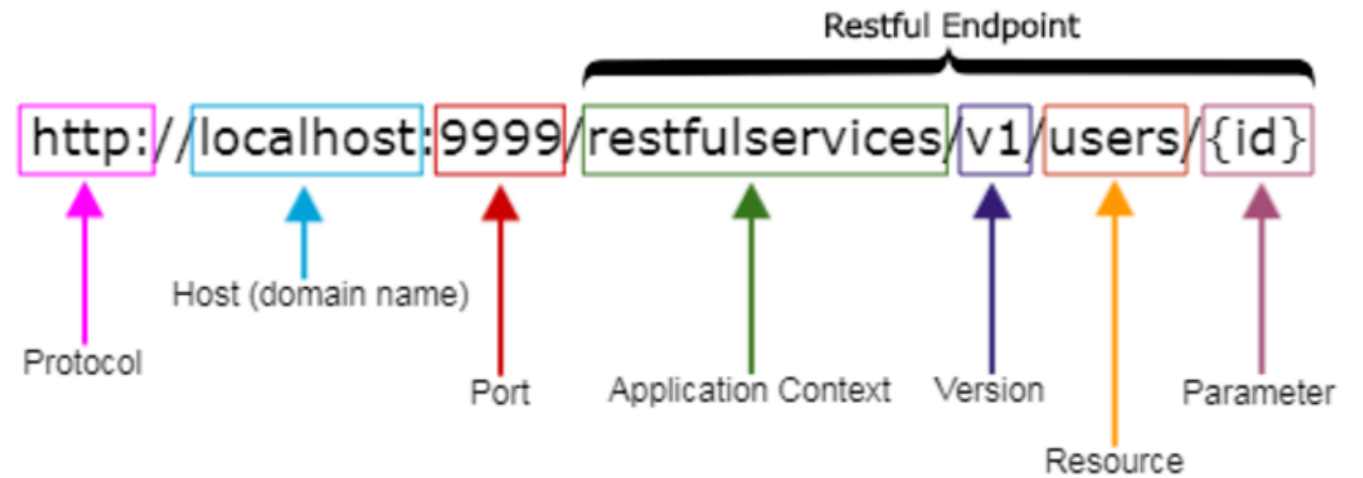
Propiedades de Calidad que REST considera

- **Evolución**
 - Cambios sin que afecten al sistema
- **Extensibilidad**
 - Añadir funcionalidad al sistema dinámicamente (cuando el sistema está deployado)
- **Personalización**
 - Especializar temporalmente la conducta de un elemento arquitectónico (ej. Para un cliente)
- **Configurabilidad**
 - Modificación Post-Deployment de componentes o sus configuraciones
- **Reusabilidad**



¿Qué es un recurso?

- Abstracción clave de la información
- Si tiene nombre puede ser un recurso
- Debe tener una URI como ID
 - thing, thing/1...



¿Qué es una representación?

“Fotografía” de un **recurso en un momento del tiempo** en algún formato

- Puede ser texto plano, imagen, JSON, HTML, YAML, XML, etc
 - Básicamente cualquier formato, idealmente estándar
- Ejemplo - Recurso manzana:
 - /apple/1



Imagen PNG

Manzana roja brillante y jugosa con una hoja verde saliendo de su tallo

яркие и сочные красное яблоко с небольшим количеством зеленых листьев стебель из

Texto TXT

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>A random apple</title>
  </head>
  <body>
    La manzana es roja, jugosa, brillante,
    y tiene una hoja verde en el tallo.

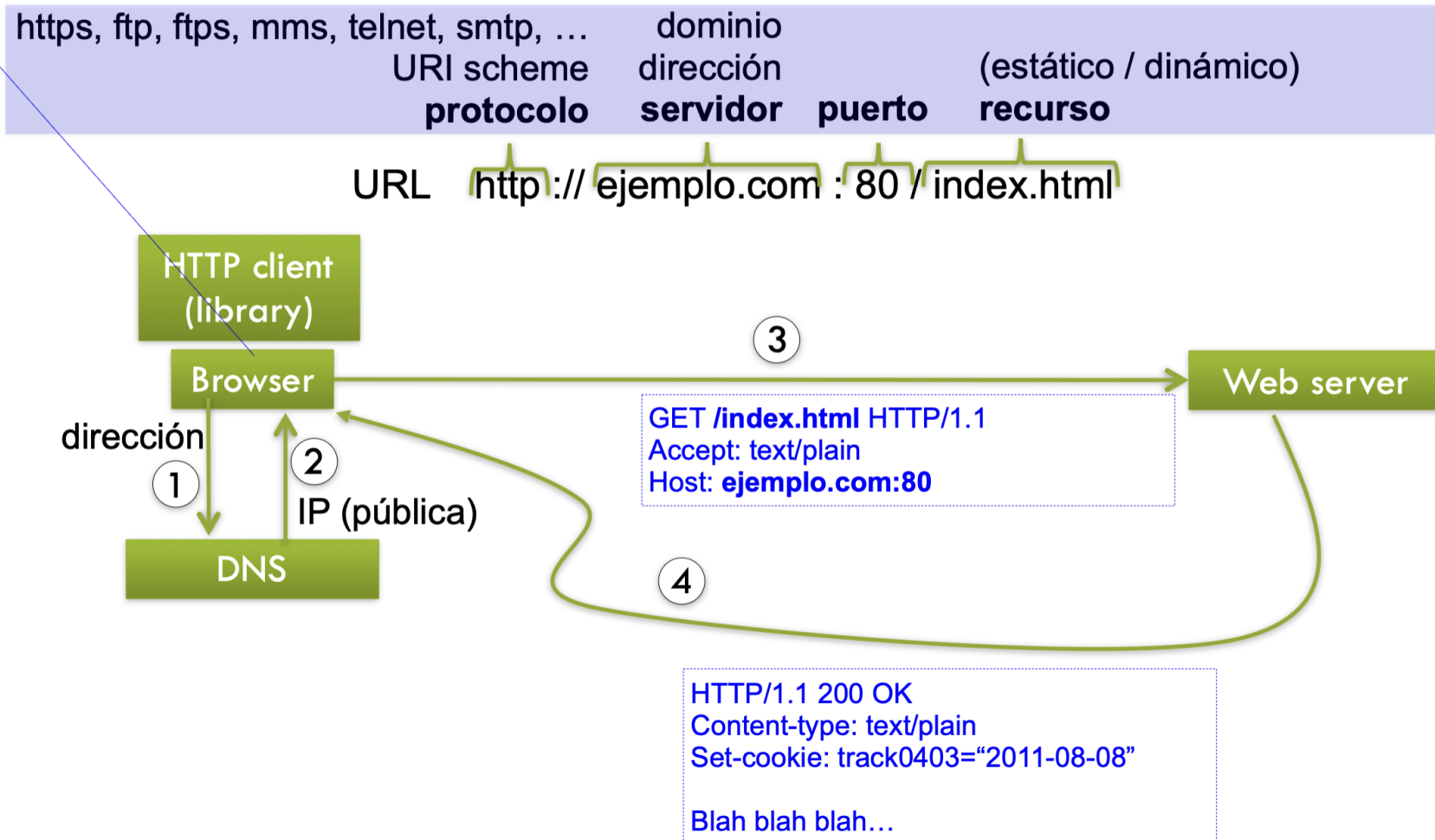
    Para ver más sobre manzanas,
    visita <a href="https://en.wikipedia.org/wiki/Apple">
  </body>
</html>
```

HTML

```
{
  "href": "/apple",
  "description": "una manzanita roja",
  "hasLeaf": true,
  "colour": "#8c3a38",
  "shinyess": 87,
  "juicyness": 0.78
}
```

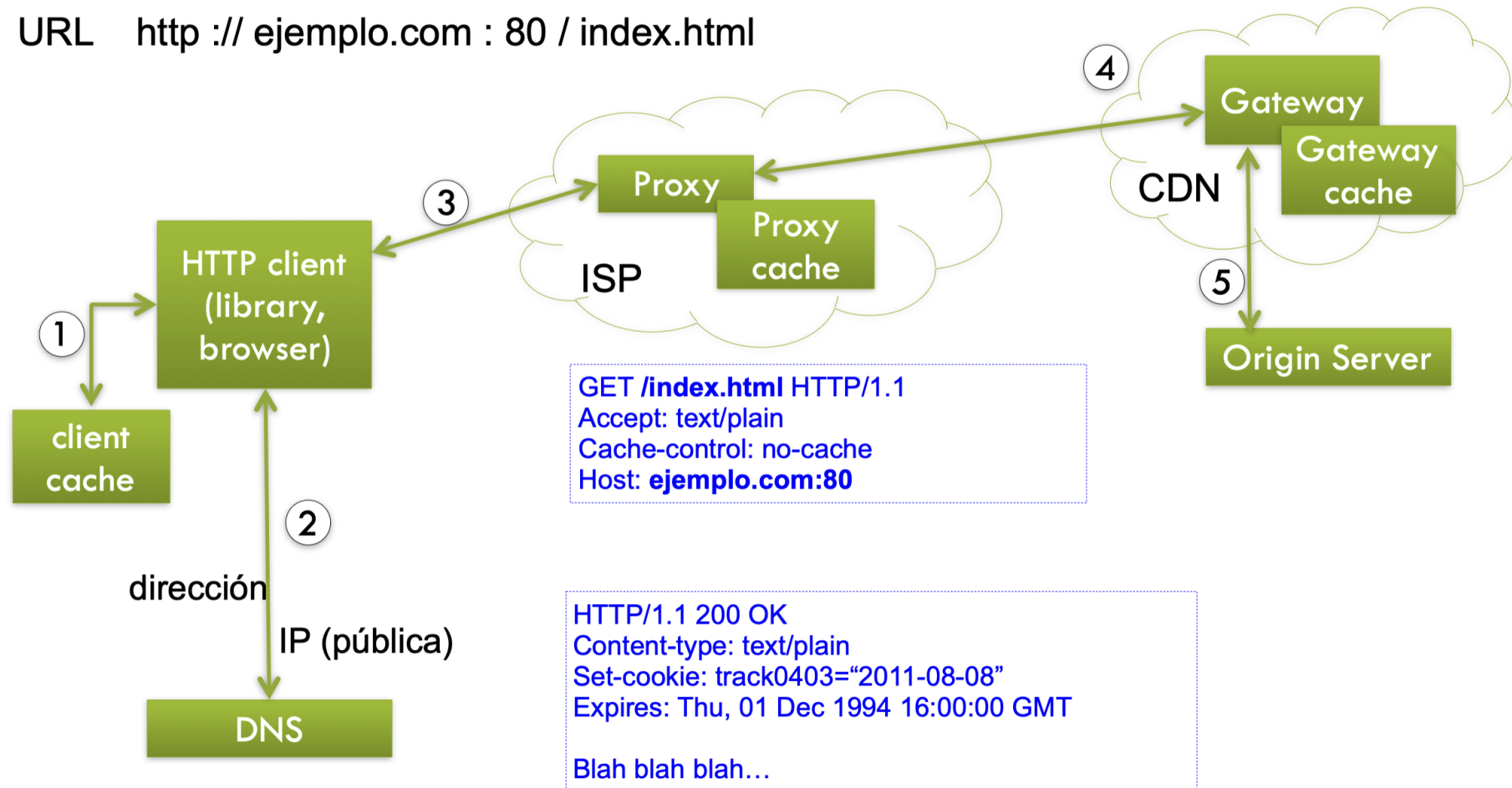
JSON

REST – Web server interaction

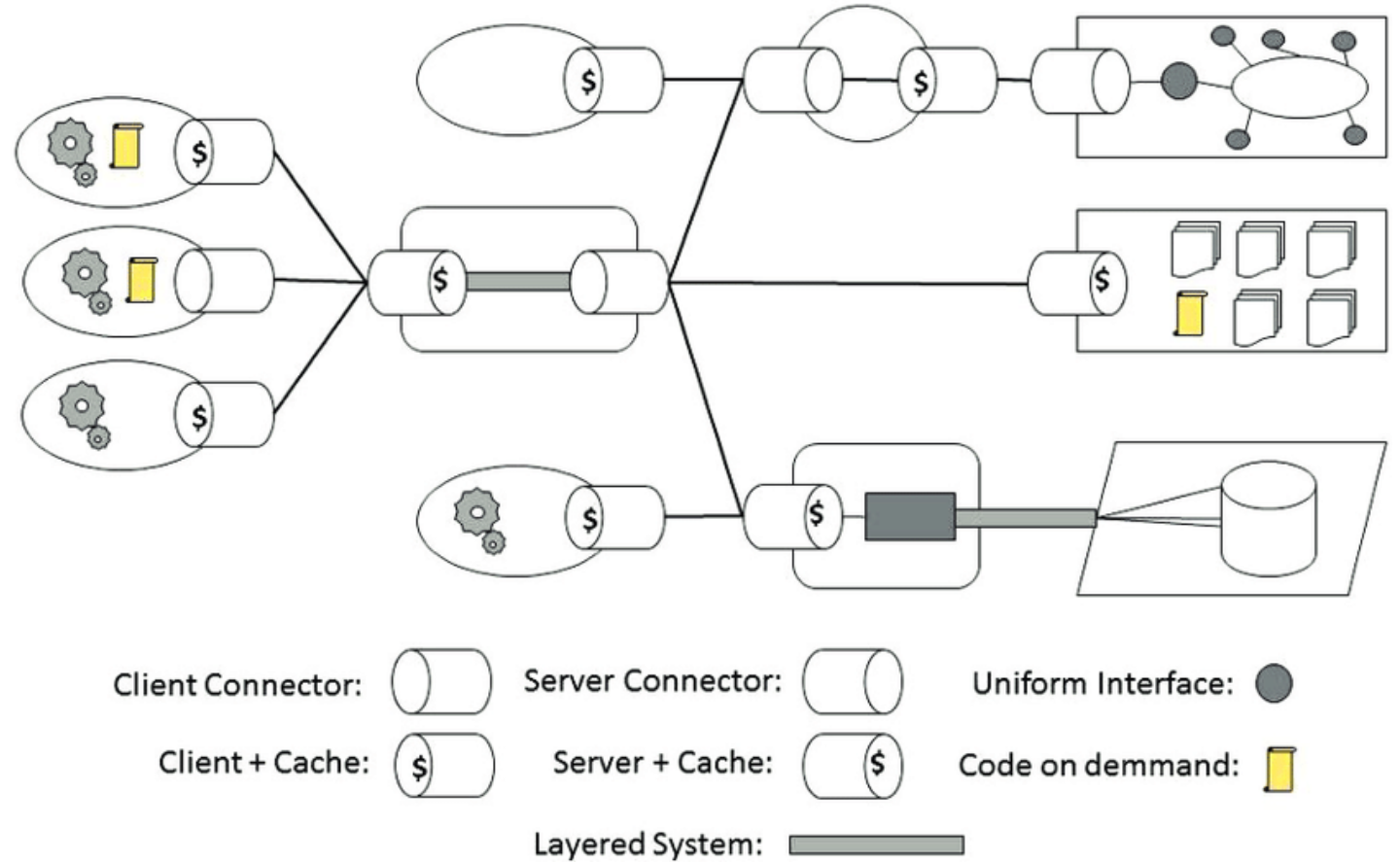


REST – Web Server Interaction

URL `http :// ejemplo.com : 80 / index.html`



REST es un estilo arquitectónico



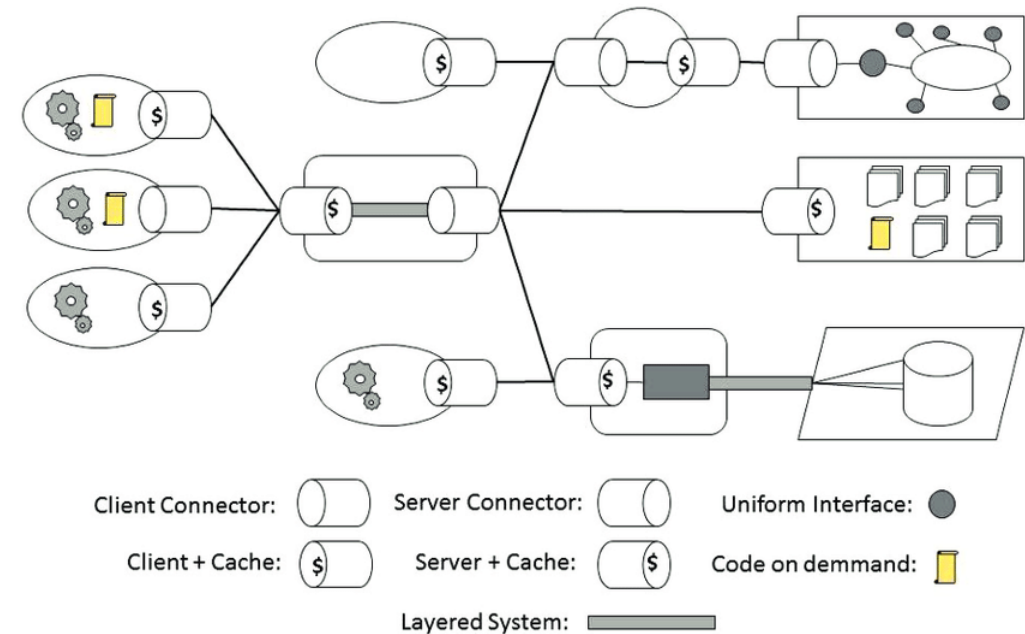
REST es un estilo arquitectónico

- **Componentes**

- **Servidor:** *Apache httpd, Microsoft IIS*
- **Gateway:** *Squid, CGI, Reverse Proxy*
- **Proxy:** *CERN Porxy, Gauntlet*
- **User Agent:** *Lynx, MOM Spider*

- **Conectores**

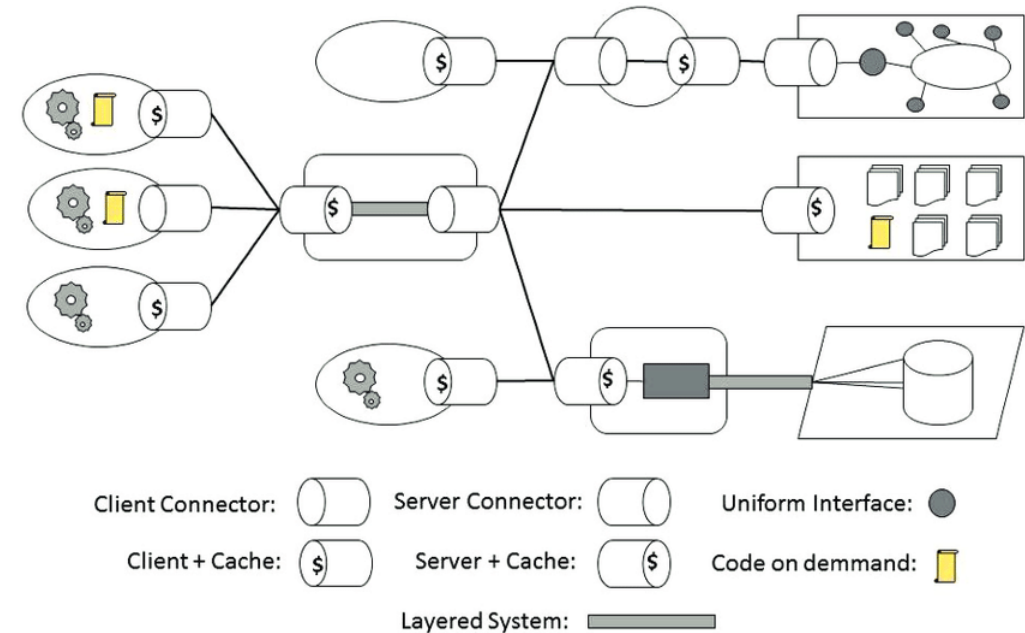
- Cliente: *libwww, libwww-perl*
- Servidor: *libwww, Apache API*
- Caché: *browser caché, Akami Caché Network*
- Resolver: *bind (DNS lookup library)*
- Tunnel: *SOCKS, SSL after HTTP Connect*



REST es un estilo arquitectónico

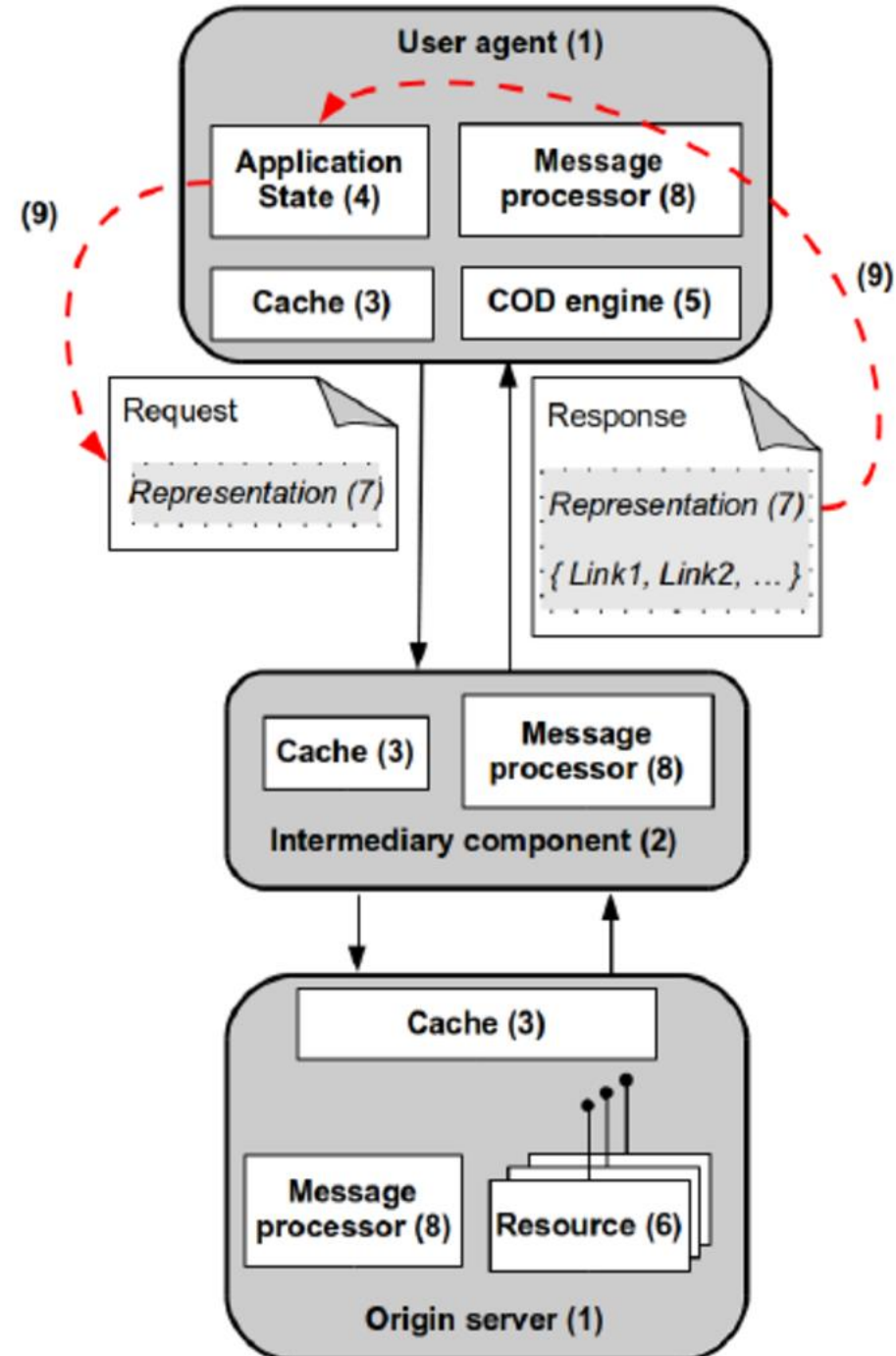
- **Elementos de datos**

- Recursos: *Objetivo conceptual*
- Identificador de recursos: *URL, URN*
- Representación: *Documento HTML, imagen JPEG, etc*
- Metadata de la representación: *media-type, last modified*
- Metadata del recurso: *source link, alternatives*
- Data de control: *if-modified-since, caché control*



Principios REST

1. Cliente/Servidor
2. Capas
3. Cacheable
4. Stateless
 - Servidor no almacena información del estado de su conversación con el cliente
 - Cada Request debe tener todo lo necesario para identificar la respuesta
 - El estado de la sesión la mantiene el cliente
 - Servidor **podría ser Stateful**

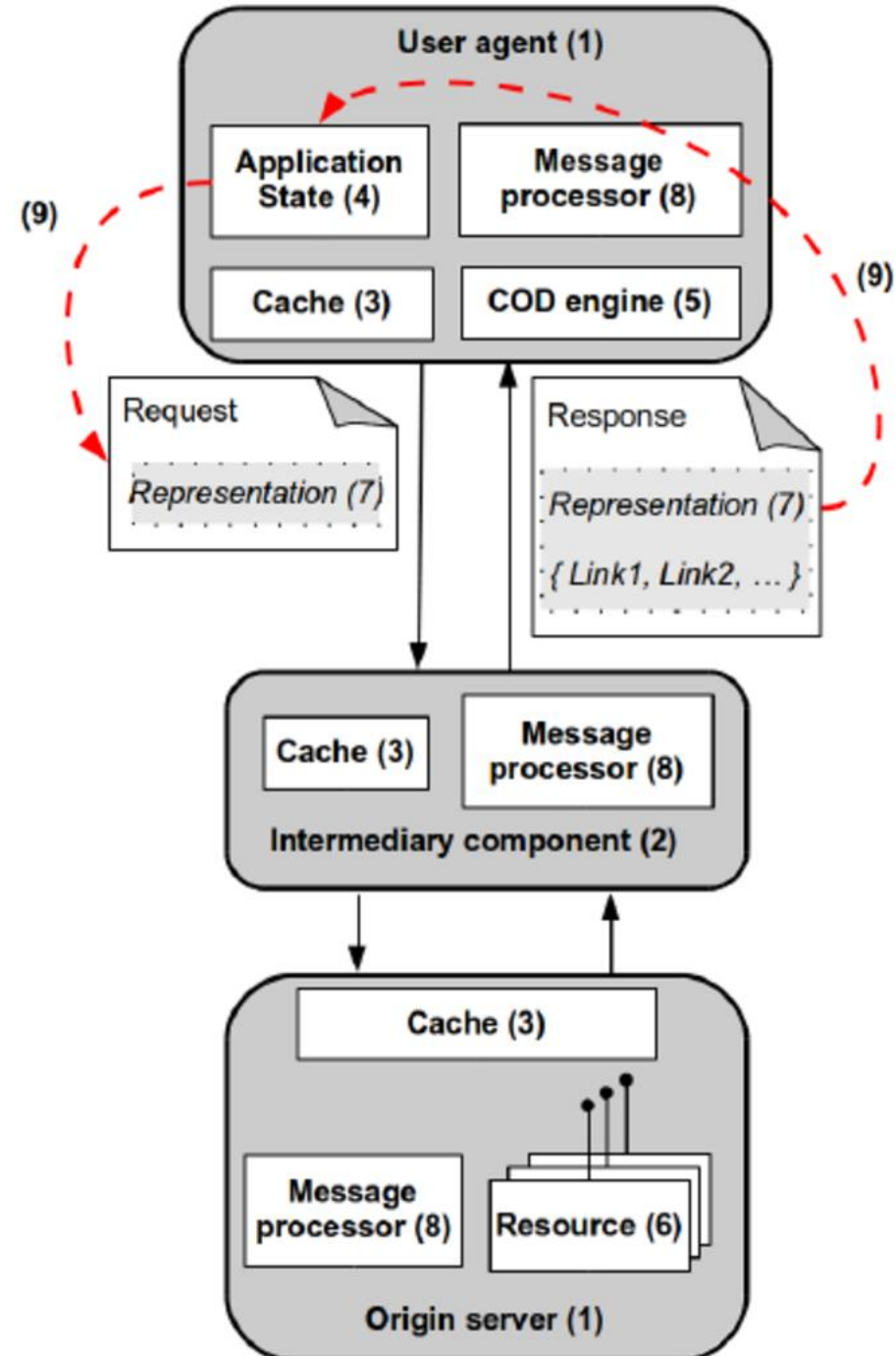


Principios REST

5. Code On Demand

Interfaz Uniforme

- 6. Recursos identificables
- 7. Manipulación mediante representaciones
- 8. Mensajes autodescriptivos
- 9. Hipermedia como Motor de estado de la aplicación



Restricciones de Interfaz Uniforme

1. Los recursos identificables son únicos y opacos
 - URI: `http://ejemplo.com/cliente/bar/orden/1`
 - Evitar acoplamiento entre clientes y servidores
 - Opaco: El cliente no debe adivinar la URI
 - Evitar el antipatrón: `http://{host}/{service}/{id}`
 - Cambio de URIs: Proxies
 - Los recursos son **conceptuales** (entidades)
 - Los recursos se sirven al cliente mediante representaciones



Restricciones de Interfaz Uniforme

2. Los recursos son conceptuales y se manipulan (se sirven al cliente) mediante (múltiples) representaciones

- Los formatos de las representaciones pueden negociarse
- Estándar para formatos:
 - Media Types o MIME (*Multipurpose Internet Mail Extensions*)
 - Cliente puede preferir JSON, XML, XHTML, en ese orden
 - Header: Accept: application/json, application/xml
 - URI: GET /algo

```
{  
  "href": "/apple",  
  "description": "una manzanita roja",  
  "hasLeaf": true,  
  "colour": "#8c3a38",  
  "shinyness": 87,  
  "juicyness": 0.78  
}
```

Restricciones de Interfaz Uniforme

3. Los mensajes son autodescriptivos y estándar

- Las operaciones HTTP tienen **semánticas conocidas que afectan o no el estado de los recursos**
- Dependiendo del protocolo:
 - Ej. HTTP:
 - GET: Leer (cacheable, safe)
 - POST: Crear entidad sin saber su ID (**unsafe ni idempotente**)
 - PUT: Actualizar entidad (**idempotente**)
 - DELETE: Borrar (**idempotente**)
 - 200: OK
 - 201: Created

1XX Informational Requests	100 Continue 101 Switching Protocols 102 Processing
2XX Successful Requests	200 OK 201 Created 202 Accepted 203 Non-Authoritative Information 204 No Content 205 Reset Content 206 Partial Content 207 Multi-Status 208 Already Reported
3XX Redirects	300 Multiple Choices 301 Moved Permanently 302 Found 303 See Other 304 Not Modified 305 Use Proxy 307 Temporary Redirect 308 Permanent Redirect
4XX Client Errors	400 Bad Request 401 Unauthorized 402 Payment Required 403 Forbidden 404 Not Found 405 Method Not Allowed 407 Proxy Authentication Required 408 Request Timeout 409 Conflict 410 Gone 412 Precondition Failed 416 Request Range Not Satisfiable 417 Expectation Failed 422 Unprocessable Entity 423 Locked 424 Failed Dependency 426 Upgrade Required 429 Too Many Requests 431 Request Header Fields Too Large 451 Unavailable for Legal Reasons
5XX Server Errors	500 Internal Server Error 501 Not Implemented 502 Bad Gateway 503 Service Unavailable 504 Gateway Timeout 505 HTTP Version Not Supported 506 Variant Also Negotiates 507 Insufficient Storage 508 Loop Detected 510 Not Extended 511 Network Authentication Required

Restricciones de Interfaz Uniforme

4. HATEOAS: Hypertext As The Engine Of Application State

- Las representaciones contienen los links necesarios para navegar, descubrir otros recursos, y los controles para cambiar su estado



```
<orden self="http://ejemplo.com/orden/101230">
  <cliente ref="http://ejemplo.com/cliente/bar"/>
  <producto ref="http://ejemplo.com/producto/21034"/>
  <cantidad valor="1"/>
</orden>
```

```
PUT <link rel="service.put" href="http://example.org/blog1"/>
POST <form ... method="post"> ... <input type="submit" ...>
```

Patrón: Contenedor – Item (CRUD)

- Listar contenido: **GET** /contenedor
- Añadir Item: **POST** /contenedor
 - El item va en el body
 - URI del item retornado en el header HTTP response:
 - Ej. **http://host/contenedor/item**
- Leer item: **GET** /contenedor/**item**
- Modificar item: **PUT** /contenedor/**item**
- Borrar item: **DELETE** /contenedor/**item**



¿Qué no es REST?

- No es un estándar de APIs Web
- No son APIs que retornan JSON
- No es un estándar para nombrar o formar URI
 - No promueve URIs anidadas ni índices incrementales
- No es lo mismo que CRUD
- No está ligada a HTTP

Malas prácticas

- Incluir el **método en la URI**:
 - `http://host.com/servicio/versión/recurso?method=get`
 - `http://host.com/servicio/versión/get_recurso`
- Incluir el **formato en la URI**:
 - `http://host.com/servicio/versión/recurso.json`
- **URIs no opacas** (URI-template):
 - `http://host.com/servicio/versión/{id}`
 - `http://host.com/servicio/versión/1`

Niveles de madurez de REST

Nivel 0 – No es REST: El pantano de POX (*Plain Old XML*)

- Intercambio **depende del contenido**
- **Mal uso de los verbos**
- No hay recursos diversos
 - **URI es un único endpoint** (Escucha todas las operaciones)
- Media type: XHTML

POST /appointmentService HTTP/1.1
[...headers]
<openSlotRequest date = "2010-01-04"
doctor = "jones"/>

①

HTTP/1.1 200 OK

[... headers]
<openSlotList>
 <slot start = "14" end = "15">
 <doctor id = "jones"/>
 </slot>
</openSlotList>

②

POST /appointmentService HTTP/1.1
[...headers]
<appointmentRequest>
 <slot doctor="jones" start="14" end="15"/>
 <patient id="jsmith"/>
</appointmentRequest>

③

HTTP/1.1 200 OK

[... headers]
<appointmentRequestFailure>
 <slot doctor="jones" start="14" end="15"/>
 <patient id="jsmith"/>
 <reason>Slot not available</reason>
</appointmentRequestFailure>

④

Niveles de madurez de REST

Nivel 1 – No es REST: Recursos

- Los recursos se identifican con URIs separadas
- Uso mínimo de verbos

```
POST /doctors/jones HTTP/1.1
[...headers]
<openSlotRequest date = "2010-01-04"/>
```

1

```
HTTP/1.1 200 OK
[... headers]
<openSlotList>
  <slot id = "1234" doctor = "jones" start = "1400" end = "1450"/>
  <slot id = "5678" doctor = "jones" start = "1600" end = "1650"/>
</openSlotList>
```

2

```
POST /slots/1234 HTTP/1.1
[...headers]
<appointmentRequest>
  <patient id="jsmith"/>
</appointmentRequest>
```

3

Niveles de madurez de REST

Nivel 2 – No es REST: Verbos HTTP

- Recursos identificados con URIs
- Uso de verbos HTTP
- Uso de códigos de respuesta

```
HTTP/1.1 409 Conflict
[... headers]
<openSlotList>
  <slot id = "5678" doctor = "jones" start = "16" end = "17"/>
</openSlotList>
```

4a

```
GET /doctors/jones/slots?date=20100104&status=open HTTP/1.1
Host: royalhope.nhs.uk
```

1

```
HTTP/1.1 200 OK
[... headers]
<openSlotList>
  <slot id = "1234" doctor = "jones" start = "14" end = "15"/>
  <slot id = "5678" doctor = "jones" start = "16" end = "17"/>
</openSlotList>
```

2

```
POST /slots/1234 HTTP/1.1
[...headers]
<appointmentRequest>
  <patient id="jsmith"/>
</appointmentRequest>
```

3

```
HTTP/1.1 201 Created
Location: slots/1234/appointment
[... headers]
<appointment>
  <slot id = "1234" doctor = "jones" start = "14" end = "15"/>
  <patient id = "jsmith"/>
</appointment>
```

4b

Niveles de madurez de REST

Nivel 3 – Es REST: Controles Hipermediales

- Recursos identificados con URIs
- Uso de verbos HTTP
- Uso de códigos de respuesta
- Uso de recursos interlinkados
 - **rel**: “Tipo” de acción que se obtendrá al seguir el link. Si se define por el usuario es una URI
 - Puede tener valores predeterminados como “self”
 - **href**: Indica la URI del link a seguir

GET /doctors/jones/slots?date=20100104&status=open HTTP/1.1
Host: royalhope.nhs.uk

1

HTTP/1.1 200 OK

[... headers]

<openSlotList>

<slot id = "1234" doctor = "jones" start = "14" end = "15">

<link rel = "royalhope.nhs.uk/linkrels/slot/book" href = "/slots/1234"/>

</slot>

<slot id = "5678" doctor = "jones" start = "16" end = "17">

<link rel = "royalhope.nhs.uk/linkrels/slot/book" href = "/slots/5678"/>

</slot>

</openSlotList>

2

POST /slots/1234 HTTP/1.1

[...headers]

<appointmentRequest>

<patient id="jsmith"/>

</appointmentRequest>

3

Niveles de madurez de REST

Nivel 3 – Es REST: Controles Hipermediales

- Recursos identificados con URIs
- Uso de verbos HTTP
- Uso de códigos de respuesta
- Uso de recursos interlinkeados
 - **rel**: “Tipo” de acción que se obtendrá al seguir el link. Si se define por el usuario es una URI
 - Puede tener valores predeterminados como “self”
 - **href**: Indica la URI del link a seguir

HTTP/1.1 201 Created
Location: slots/1234/appointment
[... headers]
<appointment>
 <slot id = "1234" doctor = "jones" start = "14" end = "15"/>
 <patient id = "jsmith"/>
 <link rel = "royalhope.nhs.uk/linkrels/appointment/cancel"
 href = "/slots/1234/appointment"/>
 <link rel = "royalhope.nhs.uk/linkrels/appointment/addTest"
 href = "/slots/1234/appointment/tests"/>
 <link rel = "self"
 href = "/slots/1234/appointment"/>
 <link rel = "royalhope.nhs.uk/linkrels/appointment/changeTime"
 href = "/doctors/mjones/slots?date=20100104@status=open"/>
 <link rel = "royalhope.nhs.uk/linkrels/appointment/updateContactInfo"
 href = "/patients/jsmith/contactInfo"/>
 <link rel = "royalhope.nhs.uk/linkrels/help"
 href = "/help/appointment"/>
</appointment>

Ventajas y limitaciones

- Las operaciones son estándar
 - Los clientes pueden hacer supuestos seguros
- El servidor no guarda estado de "la conversación"
 - Escalabilidad masiva
 - Caches (GET)
 - Cada mensaje debe enviar toda la información necesaria para completar la operación (¿y cuándo necesitamos contraseñas?)

Restricciones de Interfaz Uniforme

HTTP Method	RFC	Request Has Body	Response Has Body	Safe	Idempotent	Cacheable
GET	RFC 7231	No	Yes	Yes	Yes	Yes
HEAD	RFC 7231	No	No	Yes	Yes	Yes
POST	RFC 7231	Yes	Yes	No	No	No
PUT	RFC 7231	Yes	Yes	No	Yes	No
DELETE	RFC 7231	No	Yes	No	Yes	No
CONNECT	RFC 7231	Yes	Yes	No	No	No
OPTIONS	RFC 7231	Optional	Yes	Yes	Yes	No
TRACE	RFC 7231	No	Yes	Yes	Yes	No
PATCH	RFC 5789	Yes	Yes	No	No	Yes

Ventajas y limitaciones

- No hay una descripción estándar
 - El cliente puede cambiar el recurso
 - Destinado al consumo humano
 - Overhead sobre otros protocolos
 - *Follow your nose*

Restricciones de Interfaz Uniforme

HTTP Method	RFC	Request Has Body	Response Has Body	Safe	Idempotent	Cacheable
GET	RFC 7231	No	Yes	Yes	Yes	Yes
HEAD	RFC 7231	No	No	Yes	Yes	Yes
POST	RFC 7231	Yes	Yes	No	No	No
PUT	RFC 7231	Yes	Yes	No	Yes	No
DELETE	RFC 7231	No	Yes	No	Yes	No
CONNECT	RFC 7231	Yes	Yes	No	No	No
OPTIONS	RFC 7231	Optional	Yes	Yes	Yes	No
TRACE	RFC 7231	No	Yes	Yes	Yes	No
PATCH	RFC 5789	Yes	Yes	No	No	Yes

gRPC

- Implementación de **RPC de Google**
 - ¡Se integra muy bien con servicios Google!
- Usa RPC: **Llamadas como objetos nativos**
- Uso de protobuf
 - Serializador de datos
 - APIs estrictamente tipadas
 - Privilegia la **velocidad y eficiencia** (vs REST u otros)
- Transferencia binaria
 - Uso de *streams*
 - Uso de comunicación bidireccional

Update the server

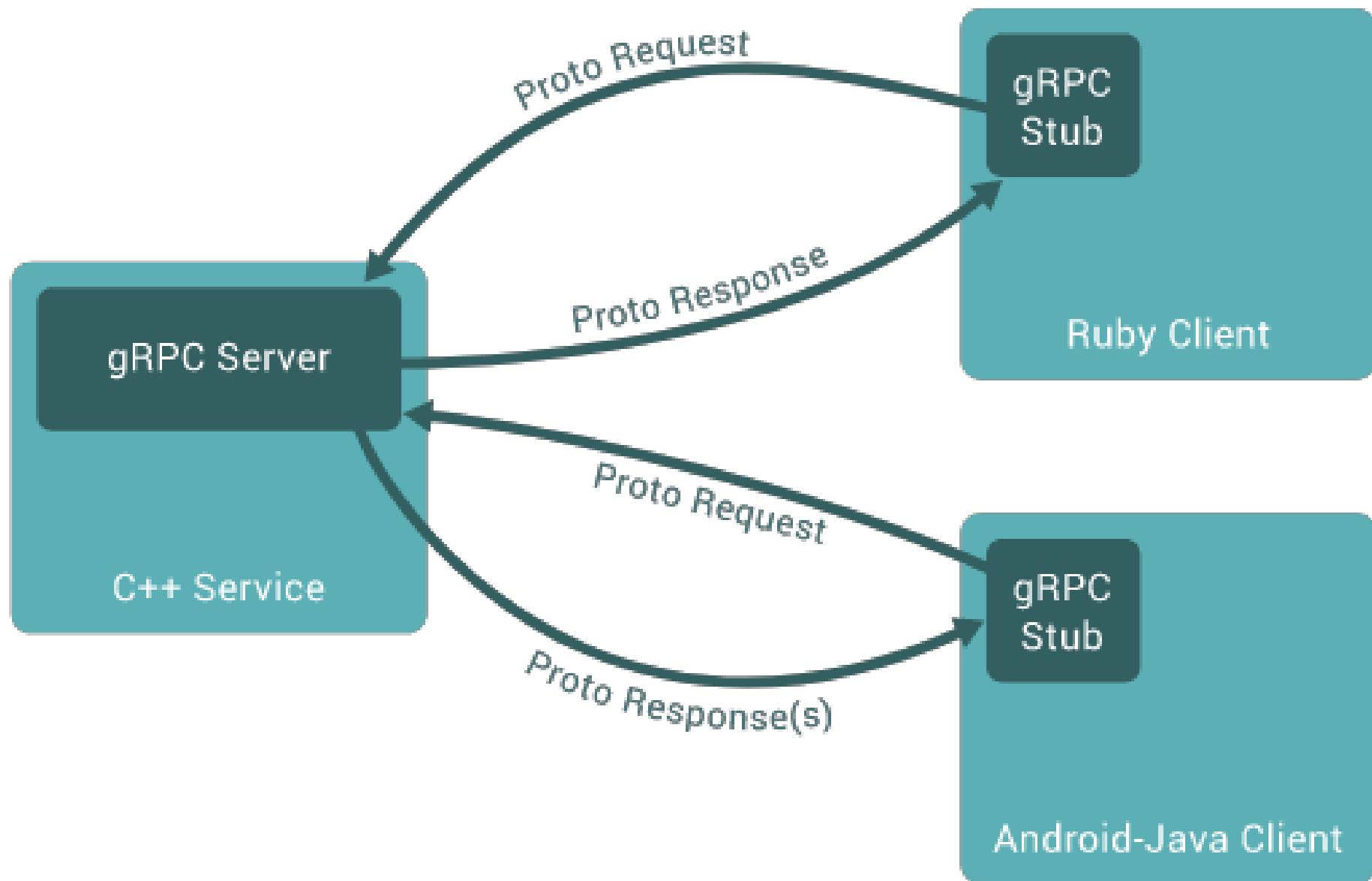
In the same directory, open `greeter_server.py`. Implement the new method like this:

```
class Greeter(helloworld_pb2_grpc.GreeterServicer):  
  
    def SayHello(self, request, context):  
        return helloworld_pb2.HelloReply(message=f'Hello, {request.name}!')  
  
    def SayHelloAgain(self, request, context):  
        return helloworld_pb2.HelloReply(message=f'Hello again, {request.name}!')  
    ...
```

Update the client

In the same directory, open `greeter_client.py`. Call the new method like this:

```
def run():  
    with grpc.insecure_channel('localhost:50051') as channel:  
        stub = helloworld_pb2_grpc.GreeterStub(channel)  
        response = stub.SayHello(helloworld_pb2.HelloRequest(name='you'))  
        print("Greeter client received: " + response.message)  
        response = stub.SayHelloAgain(helloworld_pb2.HelloRequest(name='you'))  
        print("Greeter client received: " + response.message)
```



Sobre gRPC

- Pros
 - Velocidad
 - Llamadas versátiles
 - 1-n
 - *Streaming*
 - Múltiples lenguajes soportados
- Cons
 - Atado a **RPC y no web-nativo**
 - Protocolo-no-humano (vs REST u otros)
 - Pobre compatibilidad con llamadas de gran tamaño (MB o más)
 - Requiere **curva de aprendizaje y coordinación para su uso**



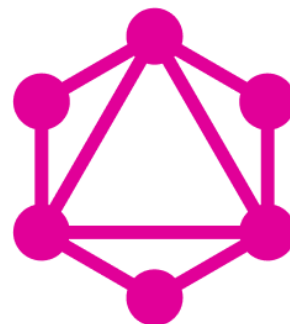
Sobre gRPC

- ¿Dónde usar?
 - Comunicación intermódulos de alta velocidad (e.g. dentro de un *bounded context* o cúmulo de microservicios)
 - Requisitos de latencia mínima
 - Necesidad de serialización eficiente



GraphQL

- API query language para llamadas definidas *client-side*
- Bien conocido, promovido inicialmente por Facebook
- Previene problemas comunes con REST/SOAP
 - Overfetching
 - Underfetching
 - Exceso de requests
 - Exceso de endpoints



GraphQL

GraphQL: Schema

- Define todas las posibles interacciones de datos con el servidor
 - Queries
 - Mutaciones
 - Suscripciones
- Básicamente “contratos”
- Federable

```
{
  hero {
    name
    friends {
      name
      homeWorld {
        name
        climate
      }
      species {
        name
        lifespan
        origin {
          name
        }
      }
    }
  }
}
```

```
type Query {
  hero: Character
}

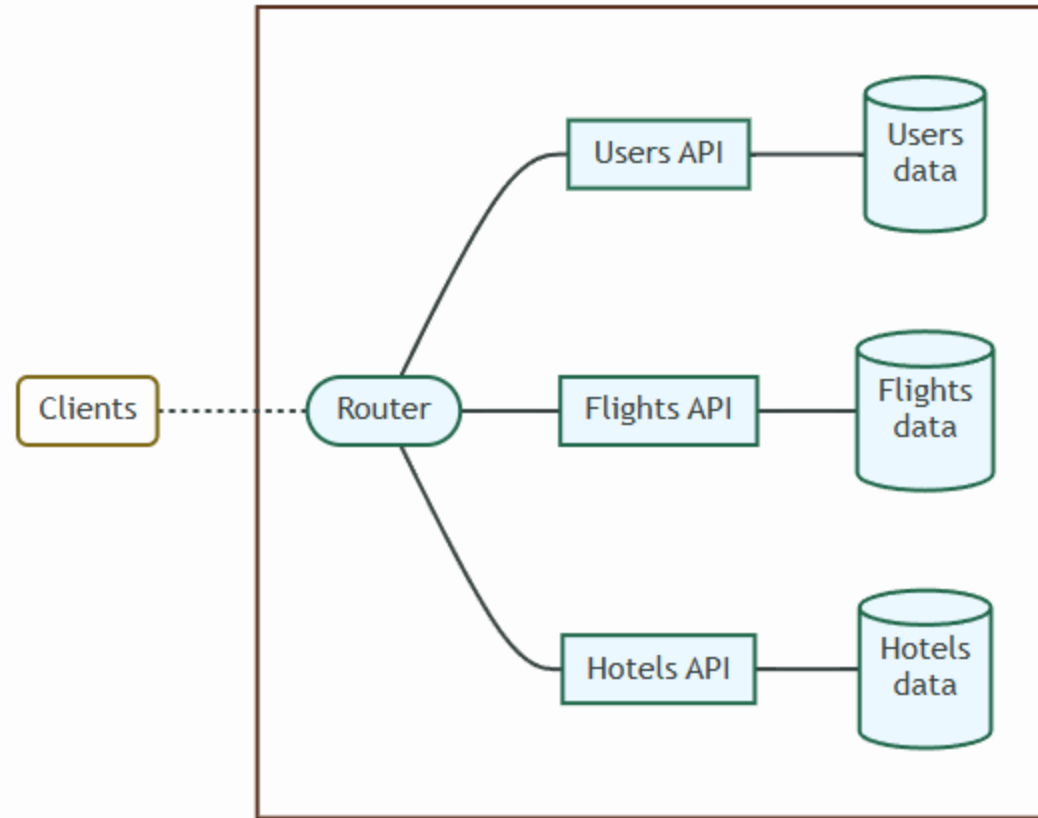
type Character {
  name: String
  friends: [Character]
  homeWorld: Planet
  species: Species
}

type Planet {
  name: String
  climate: String
}

type Species {
  name: String
  lifespan: Int
  origin: Planet
}
```

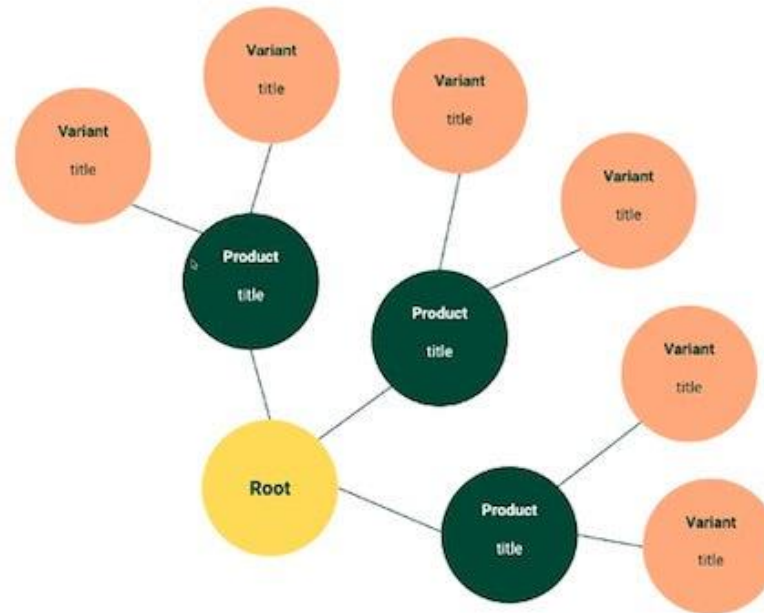
GraphQL: Schema

- Define todas las posibles interacciones de datos con el servidor
 - Queries
 - Mutaciones
 - Suscripciones
- Básicamente “contratos”
- Federable



GraphQL: Schema

- Define todas las posibles interacciones de datos con el servidor
 - Queries
 - Mutaciones
 - Suscripciones
- Básicamente “contratos”

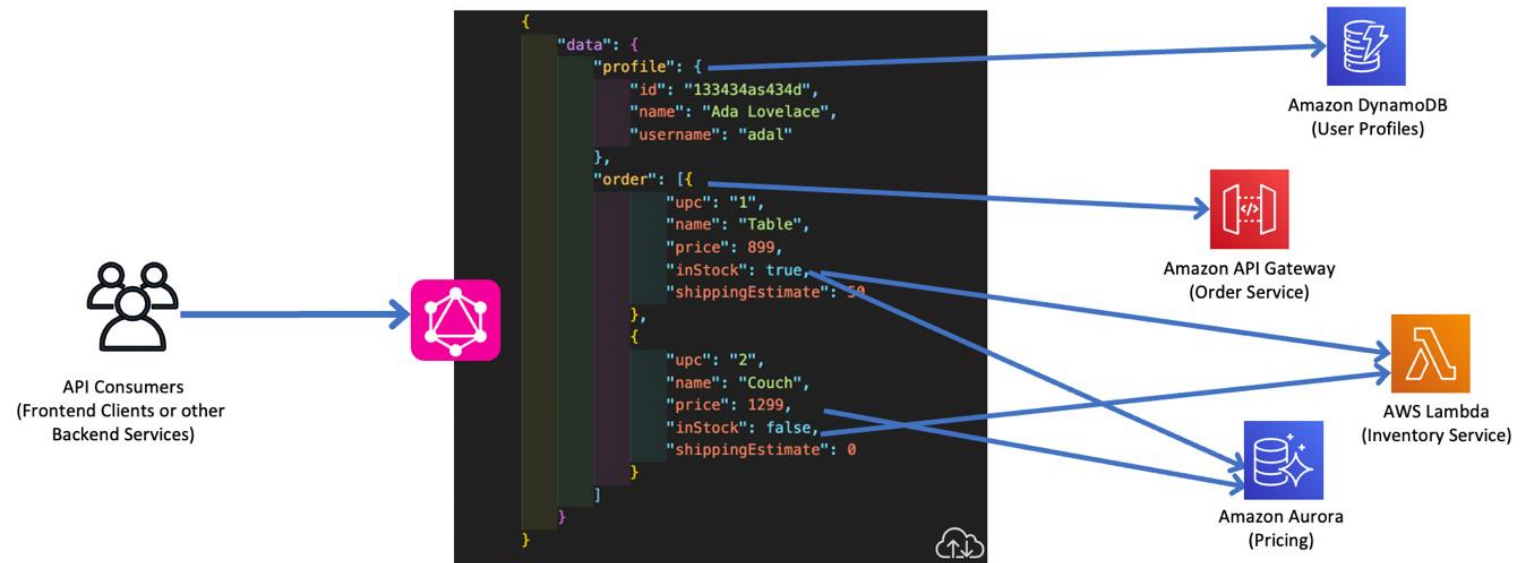


```
{
  products (first: 3) {
    edges {
      node {
        title
        variants (first:2) {
          edges {
            node {
              title
            }
          }
        }
      }
    }
  }
}
```



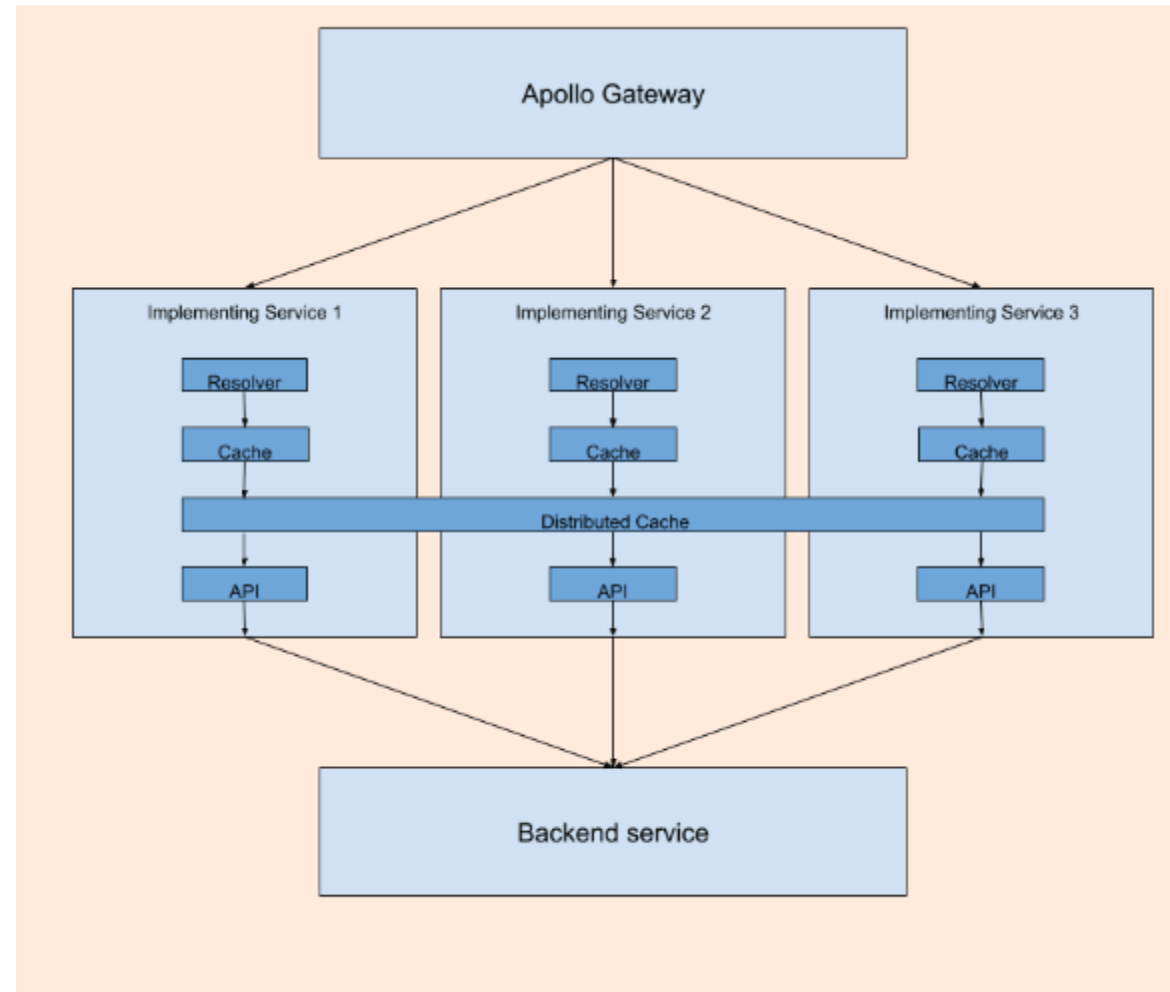
GraphQL: Pros

- **Fetching** mejorado
 - Solo un endpoint
 - Normalmente **se requiere solo una o dos peticiones**
- Capacidades de **suscripción**
 - Aplicaciones real-time
- DevEx mejorada en frontend
- **Tipado**



GraphQL: Cons

- Requiere un **gasto de desarrollo mayor** en definir los *schemas*
 - Aunque posteriormente se paga muy bien
- Curva de aprendizaje
- Caché no tan intuitivo
 - Se **pierde el apoyo de HTTP**
- Problemas de performance: **difícil de optimizar**
- Clientes mal configurados o maliciosos podrían pedir data muy compleja y extensiva en un solo *request*





Integración

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot